Exploratory Data Analysis

PSYC 259: Principles of Data Science

John Franchak & Holly O'Rourke

CDA vs. EDA

- Goals of Confirmatory Data Analysis
  - Hypothesis testing, probabilistic modeling, inference
- Goals of Exploratory Data Analysis (Tukey)
  - Understanding the patterns in the data
  - Generating hypotheses
  - Checking your assumptions about data quality
  - “To find the unexpected, to avoid being fooled, and to develop rich descriptions” (Behrens & Yu, 2003)

Behrens & Yu, 2013

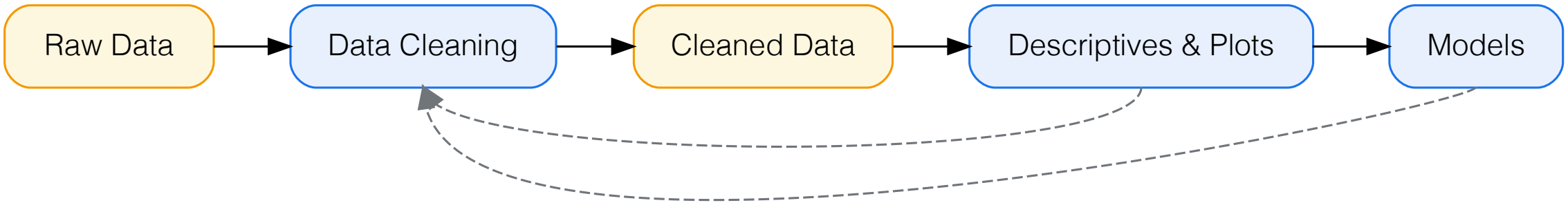
Why do we need EDA?

- Summarizing = a loss of information
  - If you first look at summarized data (across trials, across participants, etc.), you may miss important patterns that exist at the raw data level
- Statistics without graphics can be misleading
  - Correlations without looking at the scatterplot
  - Means without examining outliers/distribution
  - Statistical tests without checking *N*

Data analysis pipeline



Data analysis pipeline



Today's tutorial

- Ways to explore and check data
  - Part 1: Using tools we've already covered
  - Part 2: Using other packages: `DataExplorer`, `rstatix`
  - Part 3: Automating `ggplot2` plots to help understand your data

Follow along from the Github repo

Last updated: 2026-11-06

Part 1: EDA with tools you know

Example data for today

```
glimpse(ds)
```

```
Rows: 60
Columns: 44
$ id                     <dbl> 103, 103, 104, 104, 200, 200, 201, 201, 202, 2…
$ condition              <chr> "walk", "search", "walk", "search", "walk", "s…
$ test_date              <dttm> 2019-02-07, 2019-02-07, 2019-02-20, 2019-02-2…
$ target                 <chr> "Diamond", "Diamond", "Rectangle", "Rectangle"…
$ age                    <dbl> 21, 21, 19, 19, 19, 19, 19, 19, 19, 19, 22, 22…
$ dob                    <dttm> 1997-02-24, 1997-02-24, 1999-02-24, 1999-02-2…
$ handedness             <chr> "Right", "Right", "Right", "Right", "Right", "…
$ race                   <chr> "Other: Mexican", "Other: Mexican", "Asian", "…
$ ethnicity_hispanic     <chr> "Yes", "Yes", "No", "No", "No", "No", "No", "N…
$ sex                    <chr> "Female", "Female", "Female", "Female", "Male"…
$ normal_corrected_vision <chr> "Normal", "Normal", "Normal", "Normal", "Norma…
$ temperature            <dbl> NA, NA, 75.0, 75.0, 62.0, 62.0, 64.0, 64.0, 64…
$ humidity               <dbl> NA, NA, 0.27, 0.27, 0.22, 0.22, 0.36, 0.36, 0.…
$ len                    <dbl> 8023, 20152, 7092, 21344, 8606, 22818, 9448, 2…
$ gazex_std              <dbl> 15.5814, 21.7167, 29.3720, 30.4992, 26.3640, 3…
$ gazey_std              <dbl> 11.9732, 11.4963, 18.8811, 17.3218, 18.1747, 1…
$ posx_mean              <dbl> 8.69110, 5.59090, 9.24680, 2.84720, -1.32810, …
$ posx_std               <dbl> 6.1267, 10.2604, 20.4751, 23.6712, 15.9540, 18…
$ posx_speed             <dbl> 0.47016, 0.33660, 1.29310, 0.71103, 0.88678, 0…
$ posy_mean              <dbl> 4.19270, 5.10900, -1.57420, 1.04940, -4.84690,…
```

Things we already know how to do

- Use `glimpse()` to check that variables are correctly classed
- Use `filter()` to check for values outside expected boundaries
- Use logical statements to check assumptions
- Use `if...else` to print warnings
- Use `count()` to check frequencies
- Use `summarize()` to look at descriptive statistics

Data checks using filters

```r
# Filter based on expected ranges
ds %>% select(id, age) %>% filter(age < 18 | age > 22)
```

```
# A tibble: 2 × 2
     id   age
  <dbl> <dbl>
1   206    24
2   206    24
```

```r
# Filter based a set of possible values
ds %>% select(id, handedness) %>% filter(!(handedness %in% c("Right", "Left")))
```

```
# A tibble: 2 × 2
     id handedness
  <dbl> <chr>
1   221 <NA>
2   221 <NA>
```

Print conditional warnings and errors

```
# Filter based on expected ranges
check_age <- ds %>% select(id, age) %>% filter(age < 18 | age > 22)

if (nrow(check_age) > 0) print(str_glue("WARNING: {nrow(check_age)} participants have ages out of
```

```
WARNING: 2 participants have ages out of the expected range
```

Print conditional warnings and errors

```
# Filter based a set of possible values
check_handedness <- ds %>% select(id, handedness) %>% filter(!(handedness %in% c("Right", "Left")

if (nrow(check_handedness) > 0) {
  print("Handedness check FAILED")
  print(check_handedness)
} else {
  print("Handedness check PASSED")
}
```

```
[1] "Handedness check FAILED"
# A tibble: 2 × 2
     id handedness
  <dbl> <chr>
1   221 <NA>
2   221 <NA>
```

Checking for expected counts

```r
# Check the number of participant ids
if (length(unique(ds$id)) != 30) print("ERROR: Incomplete Dataset")

# Count
count(ds, sex)
```

```
# A tibble: 4 × 2
  sex        n
  <chr>  <int>
1 Female    22
2 Male      26
3 female     2
4 male      10
```

```r
ds$sex <- tolower(ds$sex)

# Count NAs
sum(is.na(ds$temperature))
```

```
[1] 8
```

Simple descriptive statistics

```
ds %>% drop_na(temperature) %>% summarize(temp_min = min(temperature), temp_mean = mean(temperatu
```

```
# A tibble: 1 × 3
  temp_min temp_mean temp_max
     <dbl>     <dbl>    <dbl>
1       58      72.2       92
```

```
summary(ds$temperature) #Also a nice option
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
  58.00   64.90   71.50   72.21   79.00   92.00       8
```

Part 2: EDA with other packages

Other helpful packages: rstatix

- Pipe-friendly descriptive and inferential statistics (same author as ggpubr)
- `get_summary_stats` = flexible function for descriptive stats
- `freq_table` = Frequency tables with proportion
- `identify_outliers` = detects/filters outliers
- `cor_mat` and helper functions = correlation matrices and plots

rstatix::get_summary_stats()

```
# Get summary stats
ds %>% get_summary_stats(temperature) #Note that it drops NAs
```

```
# A tibble: 1 × 13
  variable        n   min   max median    q1    q3   iqr   mad  mean    sd    se
  <fct>       <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 temperature    52    58    92   71.5  64.9    79  14.1  10.5  72.2  8.35  1.16
# ℹ 1 more variable: ci <dbl>
```

```
ds %>% group_by(target, sex) %>% get_summary_stats(temperature)
```

```
# A tibble: 4 × 15
  target   sex   variable     n   min   max median    q1    q3   iqr   mad  mean
  <chr>    <chr> <fct>    <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Diamond  fema… tempera…    16  64.9    84   75.5  72    79     7    5.19  74.7
2 Diamond  male  tempera…    10  62      83   63    63    72     9    1.48  68.6
3 Rectang… fema… tempera…     4  75      92   83.5  75    92    17   12.6   83.5
4 Rectang… male  tempera…    22  58      82   68    64.4  75.5  11.1  5.93  70.0
# ℹ 3 more variables: sd <dbl>, se <dbl>, ci <dbl>
```

rstatix::get_summary_stats()

```
# Specify multiple variables and types of output stats
ds %>% get_summary_stats(c("temperature", "humidity"), type = "common")
```

```
# A tibble: 2 × 10
  variable        n   min   max median   iqr  mean    sd    se    ci
  <fct>       <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 temperature    52 58    92     71.5  14.1  72.2  8.35  1.16  2.32
2 humidity       50  0.22  0.69   0.42  0.15  0.44 0.111 0.016 0.032
```

```
ds %>% get_summary_stats(starts_with("eyex"), type = "mean_sd")
```

```
# A tibble: 5 × 4
  variable           n   mean    sd
  <fct>          <dbl>  <dbl> <dbl>
1 eyex_mean         60 -2.87  2.64
2 eyex_std          60 12.2   2.12
3 eyex_speed        60  1.51  0.535
4 eyexpos_corr      60  0.455 0.165
5 eyexspeed_corr    60 -0.061 0.09
```

rstatix::freq_table()

```
ds %>% freq_table(sex)
```

```
# A tibble: 2 × 3
  sex        n  prop
  <chr>  <int> <dbl>
1 female    24    40
2 male      36    60
```

```
ds %>% freq_table(target, sex)
```

```
# A tibble: 4 × 4
  target    sex        n  prop
  <chr>     <chr>  <int> <dbl>
1 Diamond   female    20  66.7
2 Diamond   male      10  33.3
3 Rectangle female     4  13.3
4 Rectangle male      26  86.7
```

rstatix::identify_outliers()

```
if (nrow(identify_outliers(ds, temperature)) == 0) print("No temperature outliers")
```

```
[1] "No temperature outliers"
```

```
ds %>% identify_outliers(eyex_mean) %>% select(id, condition, eyex_mean)
```

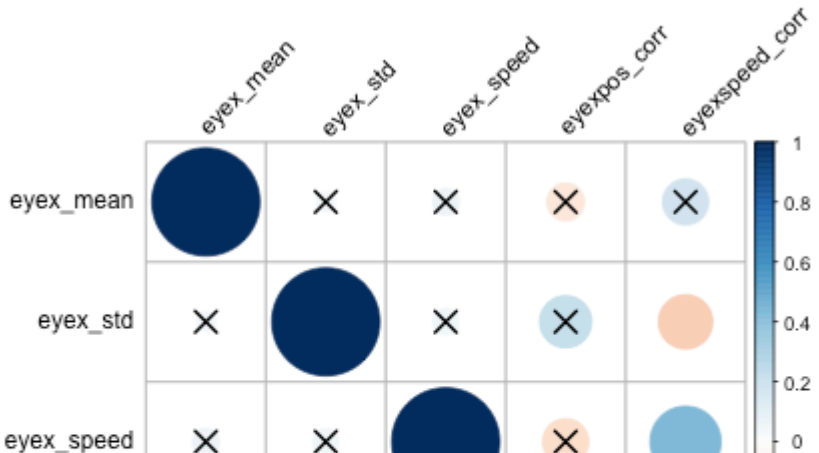
```
# A tibble: 3 × 3
     id condition eyex_mean
  <dbl> <chr>         <dbl>
1   220 walk           4.06
2   224 walk           3.42
3   227 search        -9.87
```

rstatix::cor_mat()

```
ds %>% cor_mat(starts_with("eyex_")) %>% pull_lower_triangle()
```

```
     rowname eyex_mean eyex_std eyex_speed
1  eyex_mean
2   eyex_std     0.031
3 eyex_speed      0.06    0.055
```

```
ds %>% cor_mat(starts_with("eyex")) %>% cor_plot()
```

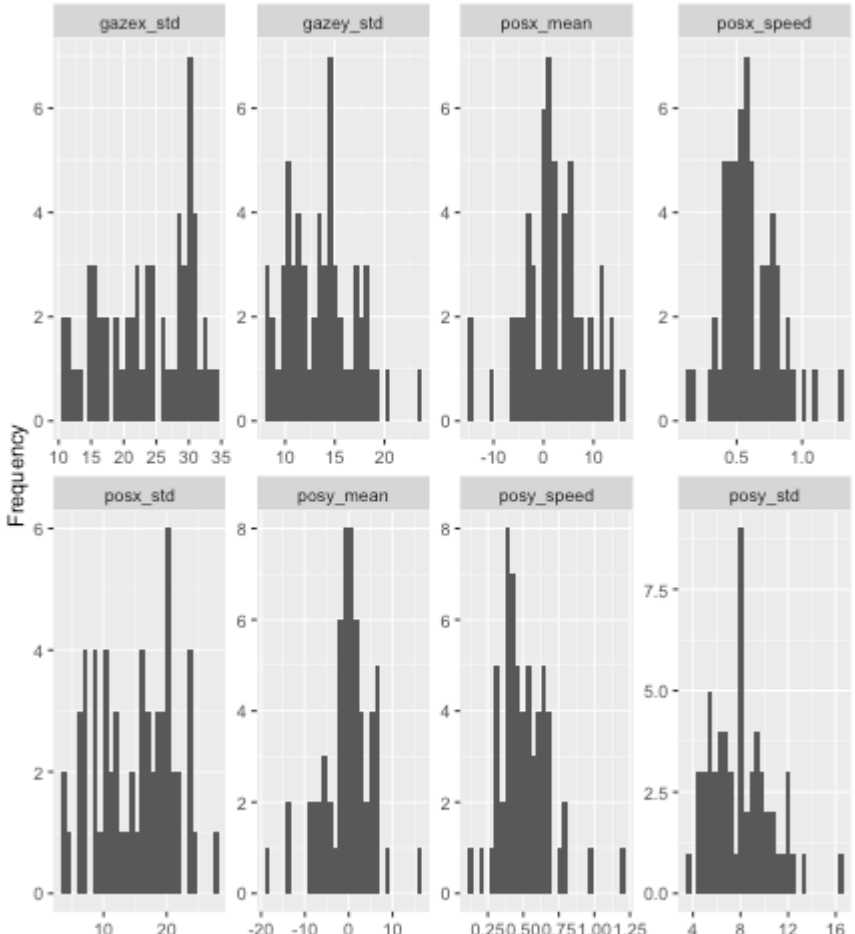


Other helpful packages: DataExplorer

- Quickly generate EDA plots for every column in your dataset
- `plot_histogram` and `plot_density` for continuous variables
- `plot_bar` for categorical
- `plot_boxplot` for detecting outliers
- `plot_missing` for missingness
- `plot_correlation` for between-column correlations at a glance
- `create_report` for everything

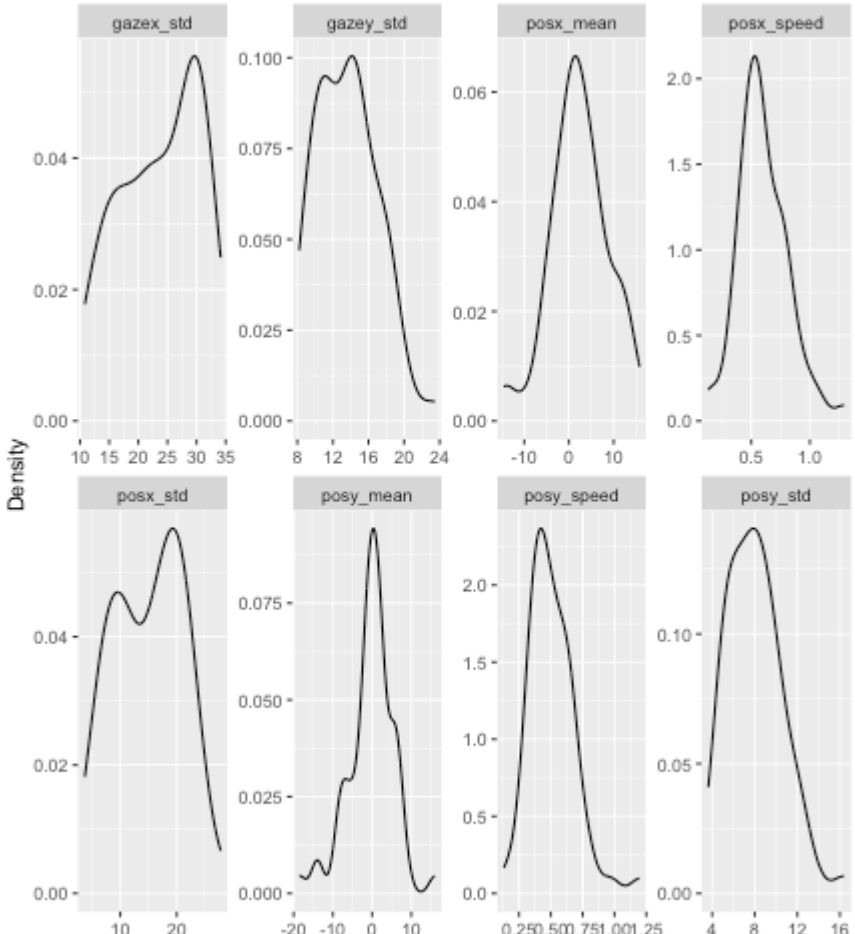
Data Explorer: Histograms

```
ds %>% select(condition, gazex_std:posy_speed) %>% plot_histogram()
```



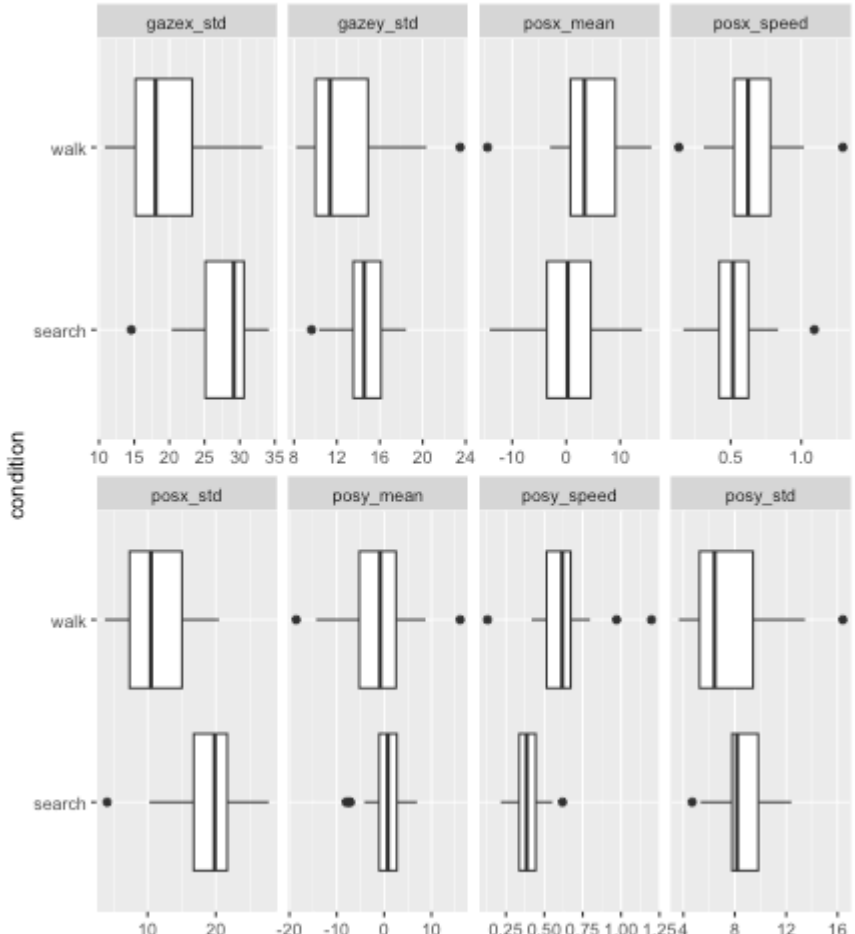
Data Explorer: Density plots

```
ds %>% select(condition, gazex_std:posy_speed) %>% plot_density()
```



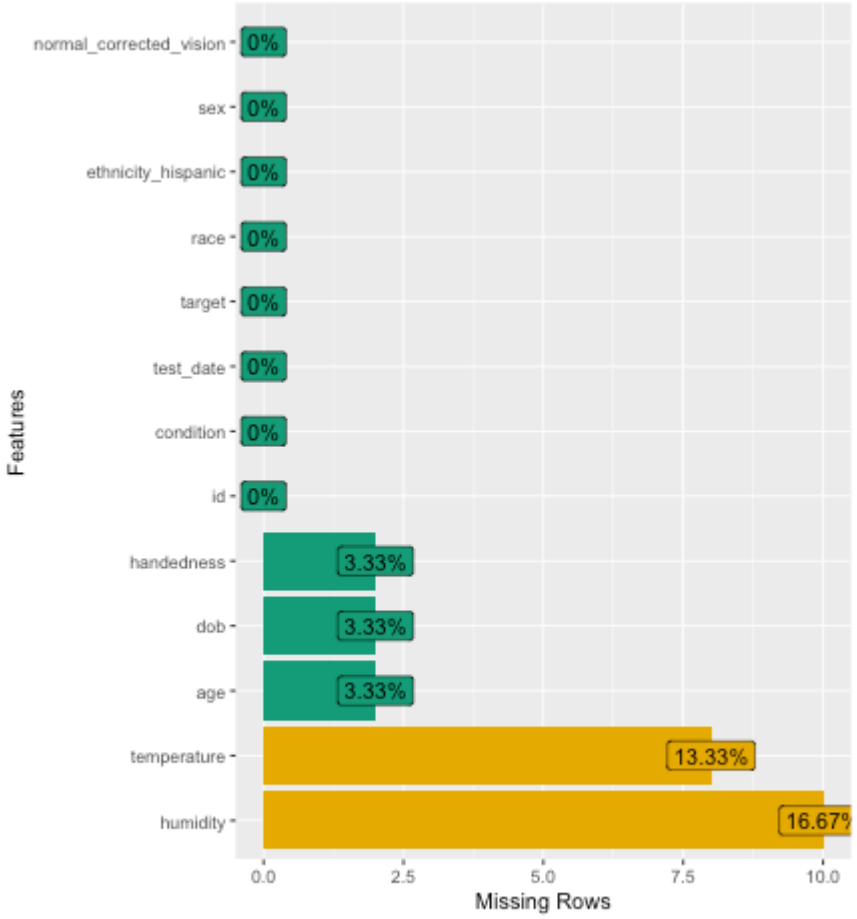
Data Explorer: Boxplots by condition

```
ds %>% select(condition, gazex_std:posy_speed) %>% plot_boxplot(by = "condition")
```



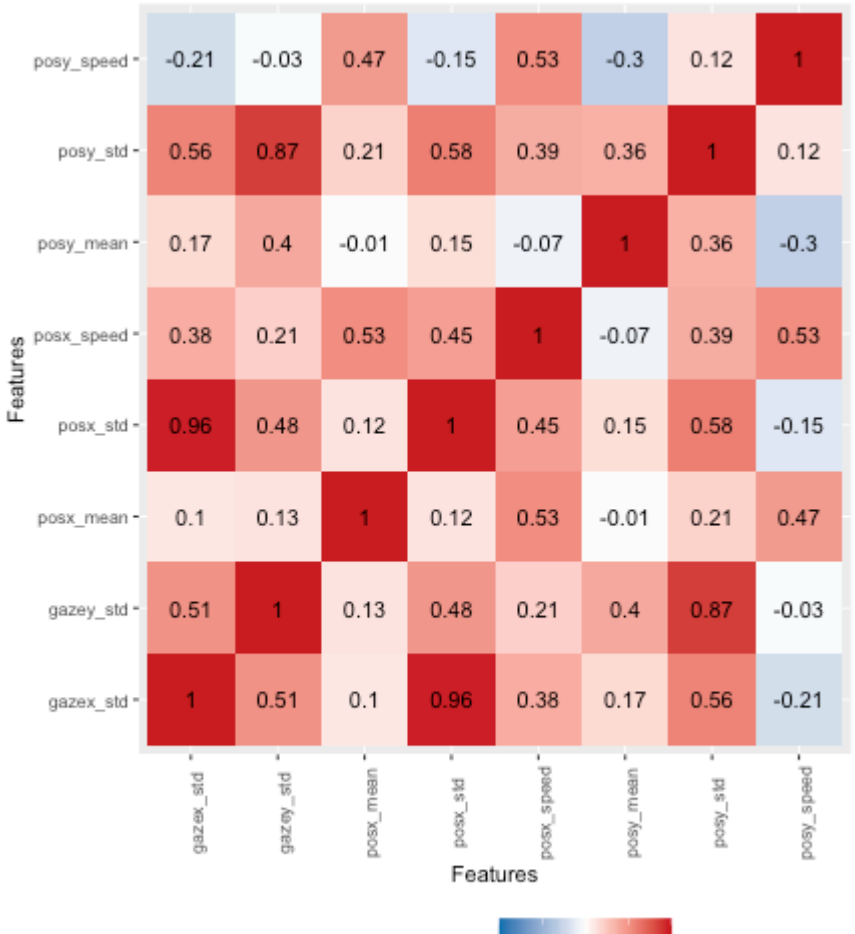
Data Explorer: Plot missing

```
ds %>% select(id:humidity) %>% plot_missing()
```



Data Explorer: Correlations

```
ds %>% select(gazex_std:posy_speed) %>% plot_correlation()
```



Data Explorer: Create Report

- `create_report()` is a brute force approach, which makes every imaginable plot

```
ds %>% create_report()
# Saves an html report to your working directory
```

Part 3: Making your own EDA plots

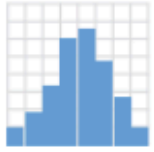
Using ggplot2

- Parts of a ggplot call:

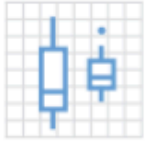
```
# Parts of a ggplot call
ggplot(DATASET, aes(MAPPING STATEMENT)) +
  geom_TYPE() +
  geom_TYPE() +
  ETC... +
  OTHER_FORMATTING() #axes, labels, legends, themes, etc.
```

- Note that ggplot uses `+`, not `%>%`

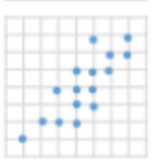
What type of aesthetic mapping depends on what type of geom



c + geom_histogram(binwidth = 5**)** x, y, alpha, color, fill, linetype, size, weight



f + geom_boxplot(), x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight



e + geom_point(), x, y, alpha, color, fill, shape, size, stroke



h + geom_bin2d(binwidth = c(0.25, 500)**)** x, y, alpha, color, fill, linetype, size, weight

What type of aesthetic mapping depends on what type of geom

- `?geom_point` and scroll down to find required aesthetics in bold

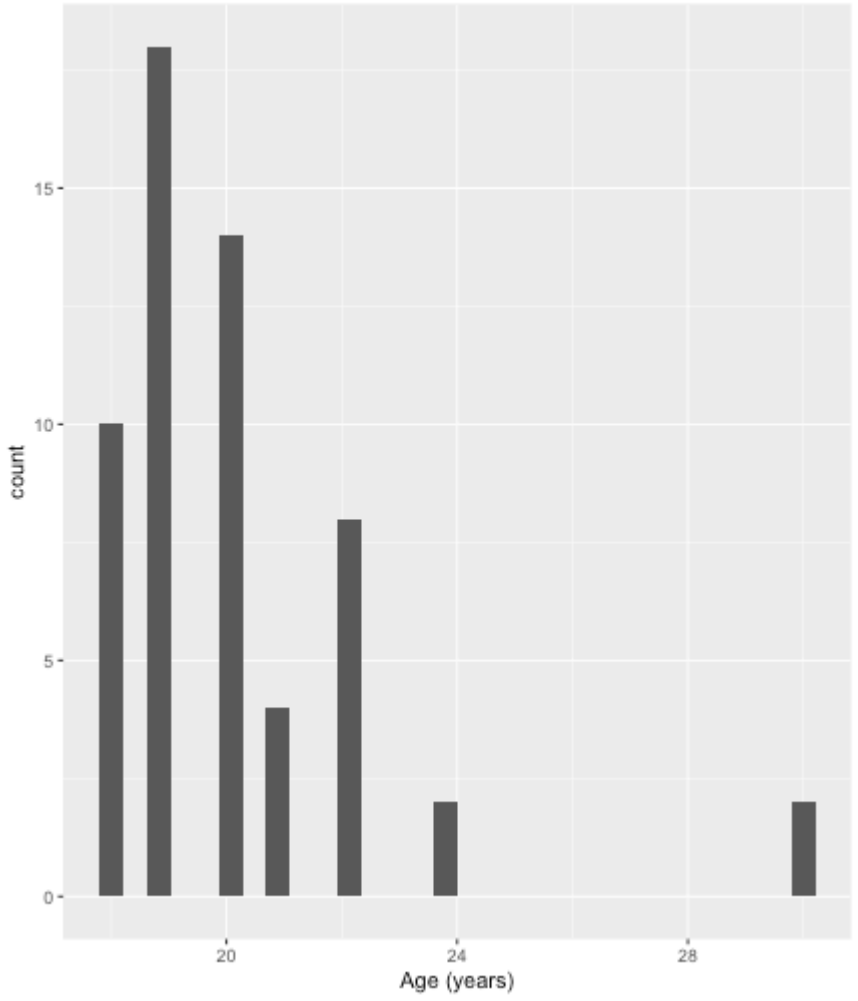
Aesthetics

`geom_point()` understands the following aesthetics (required aesthetics are in bold):

- **`x`**
- **`y`**
- `alpha`
- `colour`
- `fill`

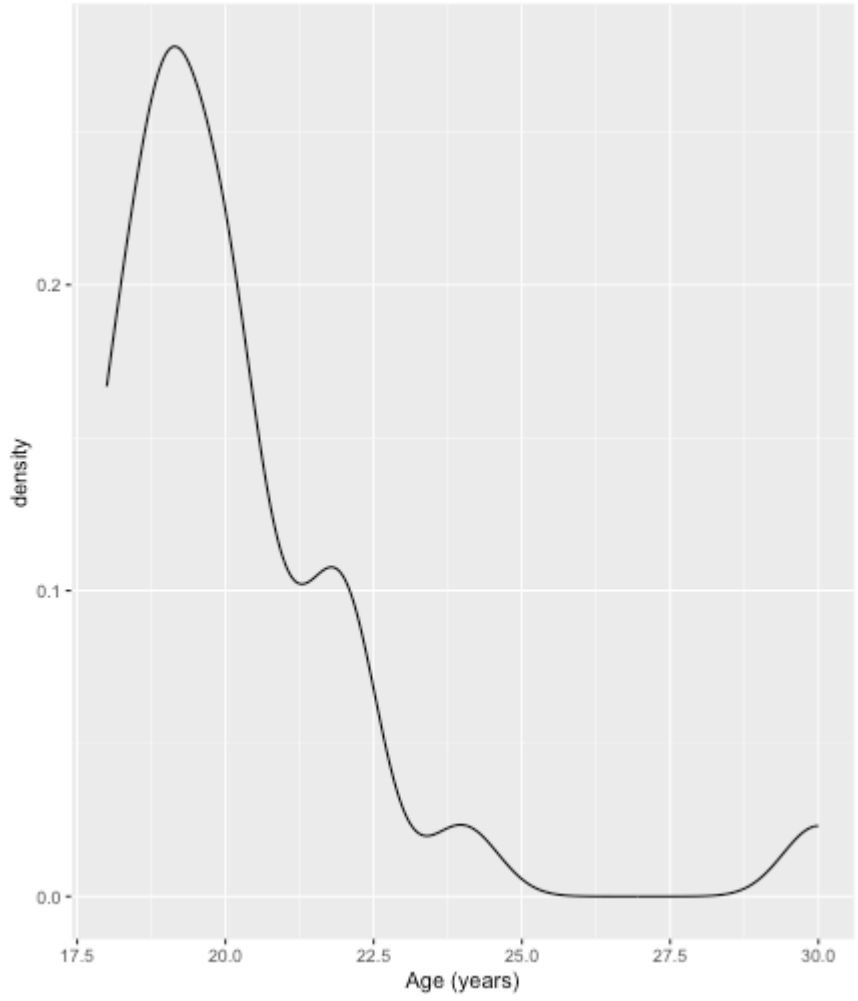
Distribution plots to check ranges

```
ds %>% mutate(age = ifelse(id == 103, 30, age)) %>%
ggplot(aes(x = age)) +
  geom_histogram() +
  xlab("Age (years)")
```



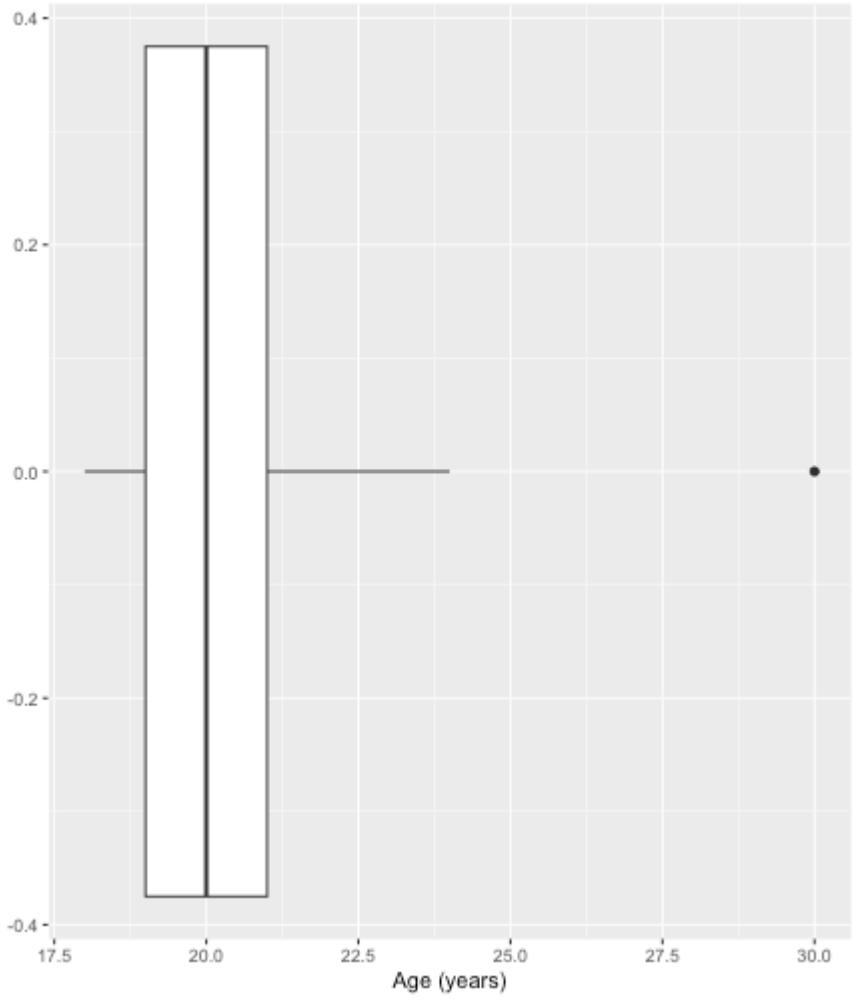
Distribution plots to check ranges

```
ds %>% mutate(age = ifelse(id == 103, 30, age)) %>%
ggplot(aes(x = age)) +
  geom_density() +
  xlab("Age (years)")
```



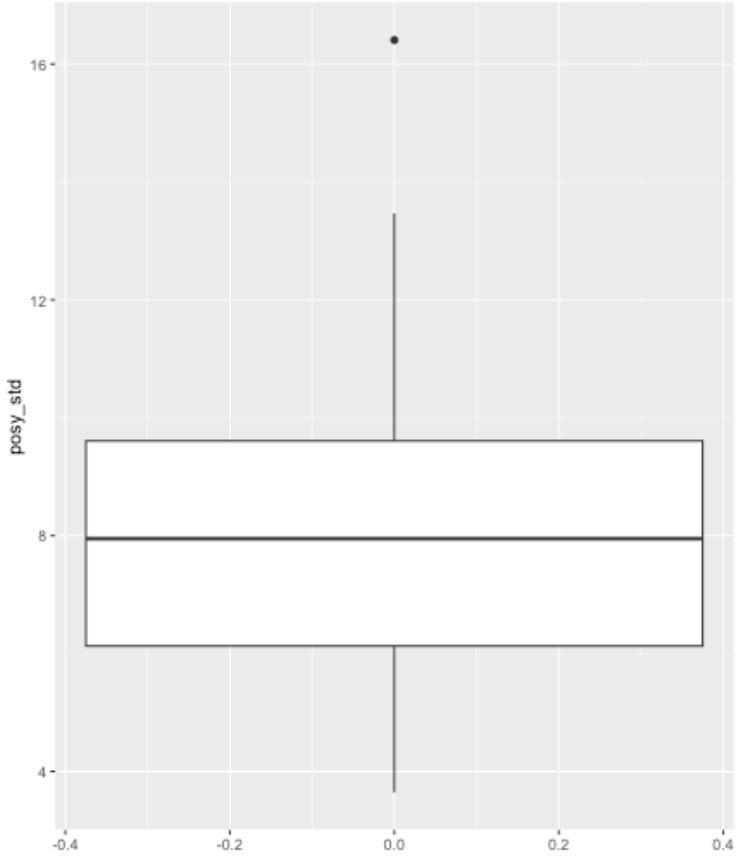
Distribution plots to check ranges

```
ds %>% mutate(age = ifelse(id == 103, 30, age)) %>%
ggplot(aes(x = age)) +
  geom_boxplot() +
  xlab("Age (years)")
```



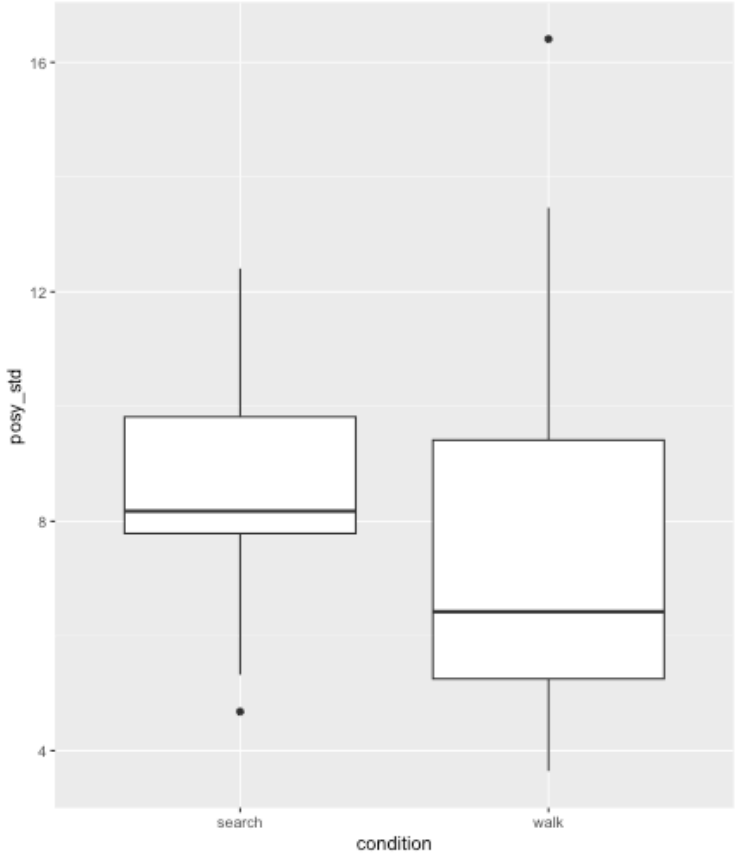
Add a category

```
ggplot(ds) +
geom_boxplot(aes(y = posy_std))
```



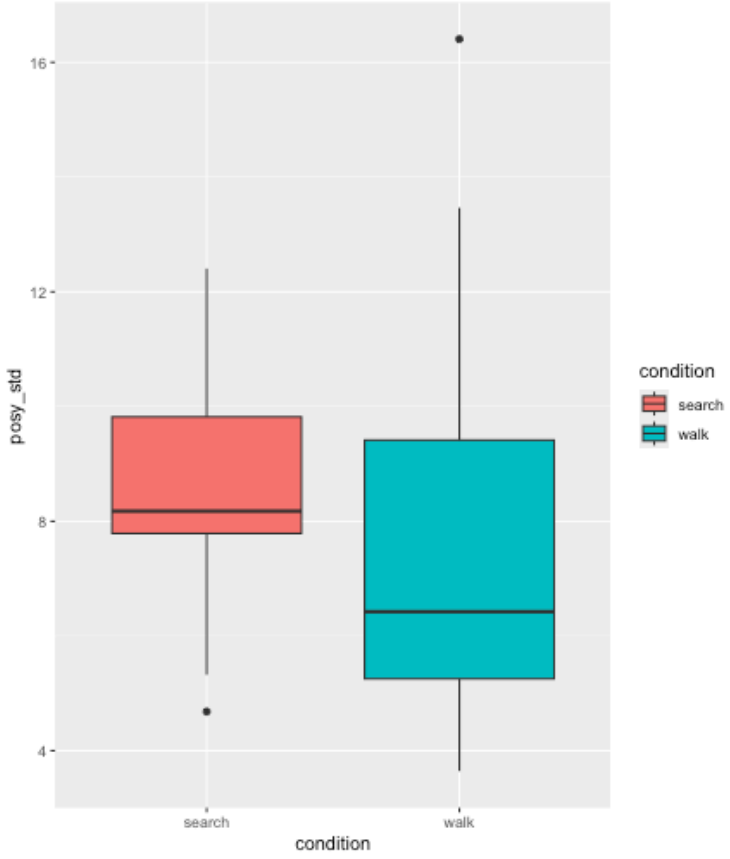
Add a category

```
ggplot(ds) +
geom_boxplot(aes(x = condition, y = posy_std))
```



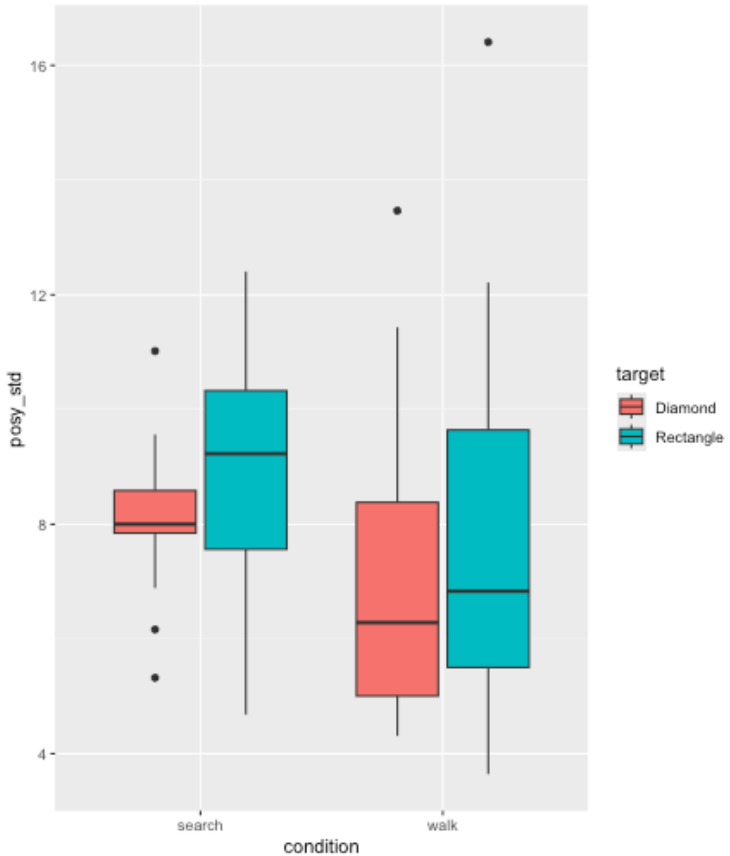
Add a category

```
ggplot(ds) +
geom_boxplot(aes(x = condition, y = posy_std, fill = condition))
```



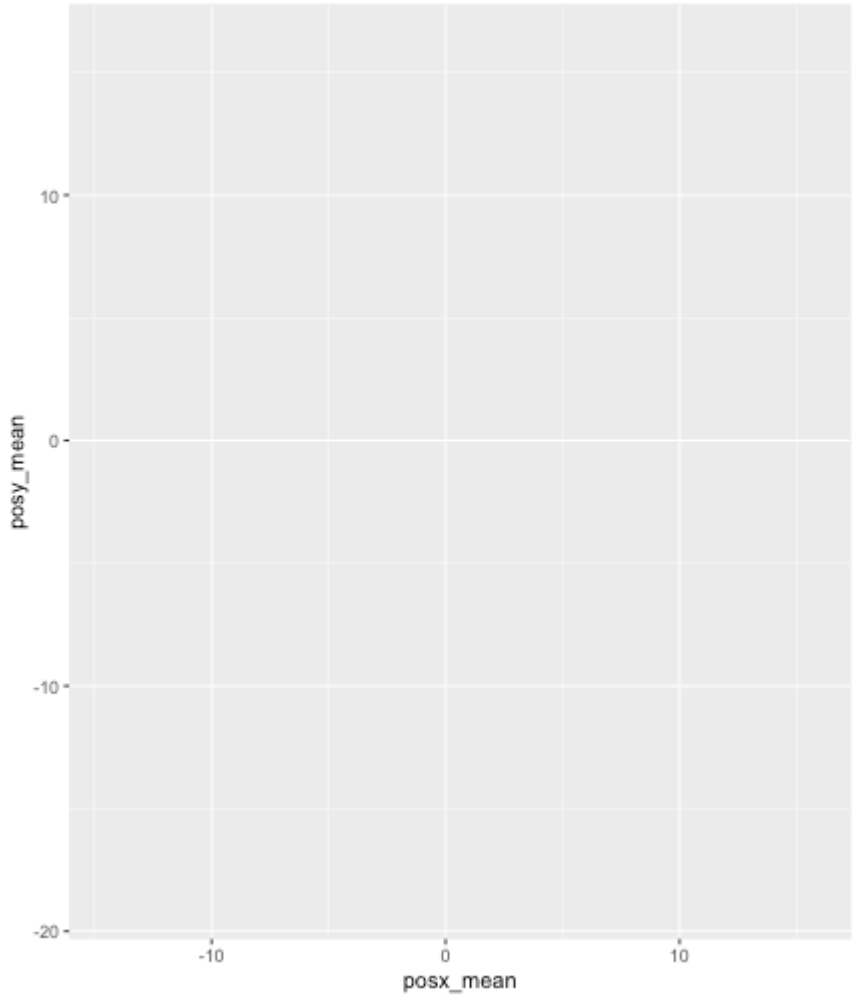
Add a category

```
ggplot(ds) +
geom_boxplot(aes(x = condition, y = posy_std, fill = target))
```



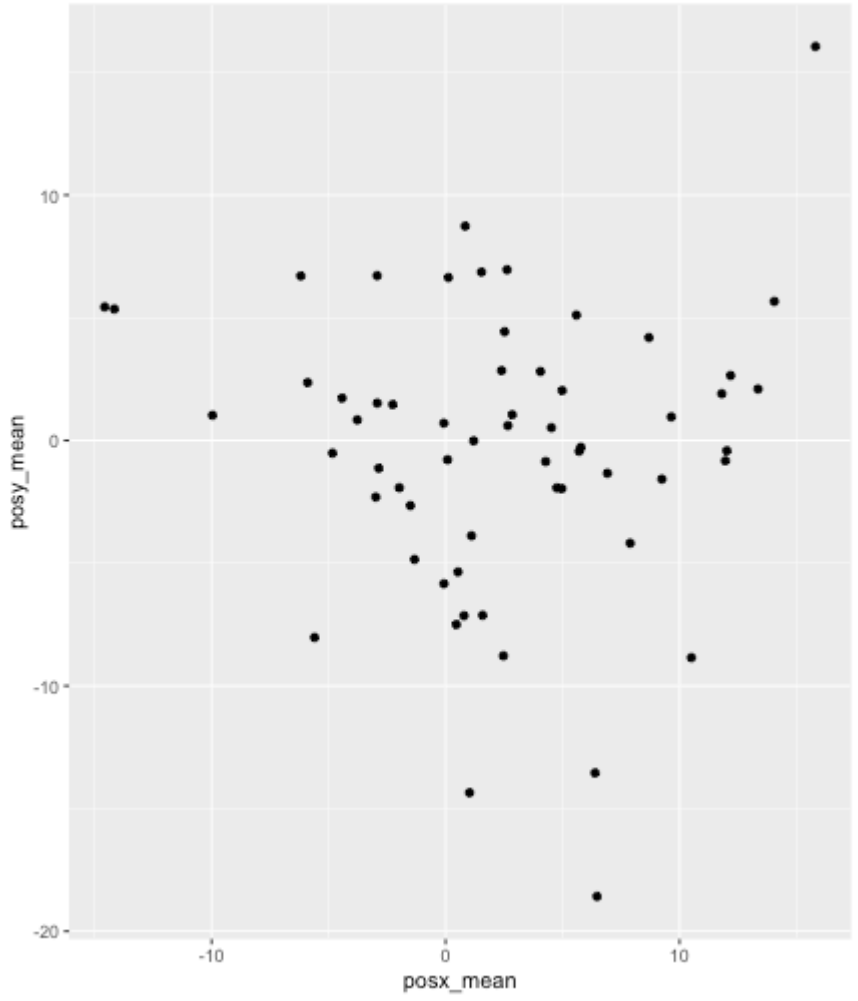
Simple scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean))
```



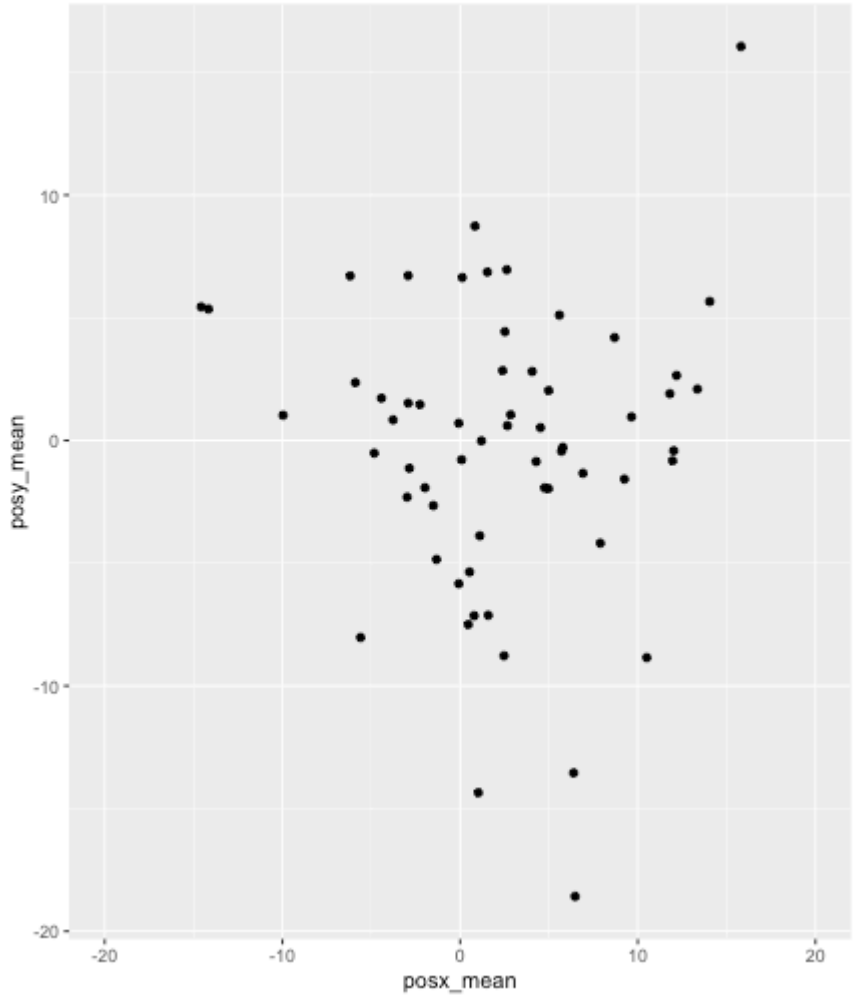
Simple scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) +
  geom_point()
```



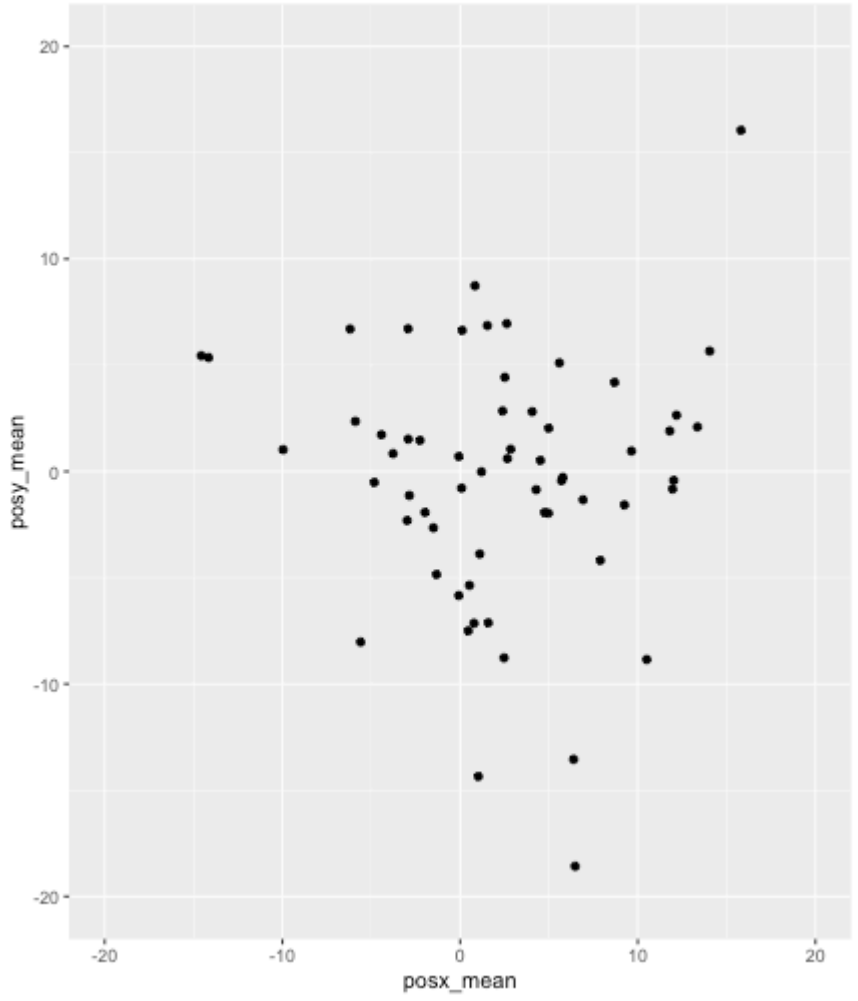
Simple scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) +
  geom_point() +
  xlim(-20, 20)
```



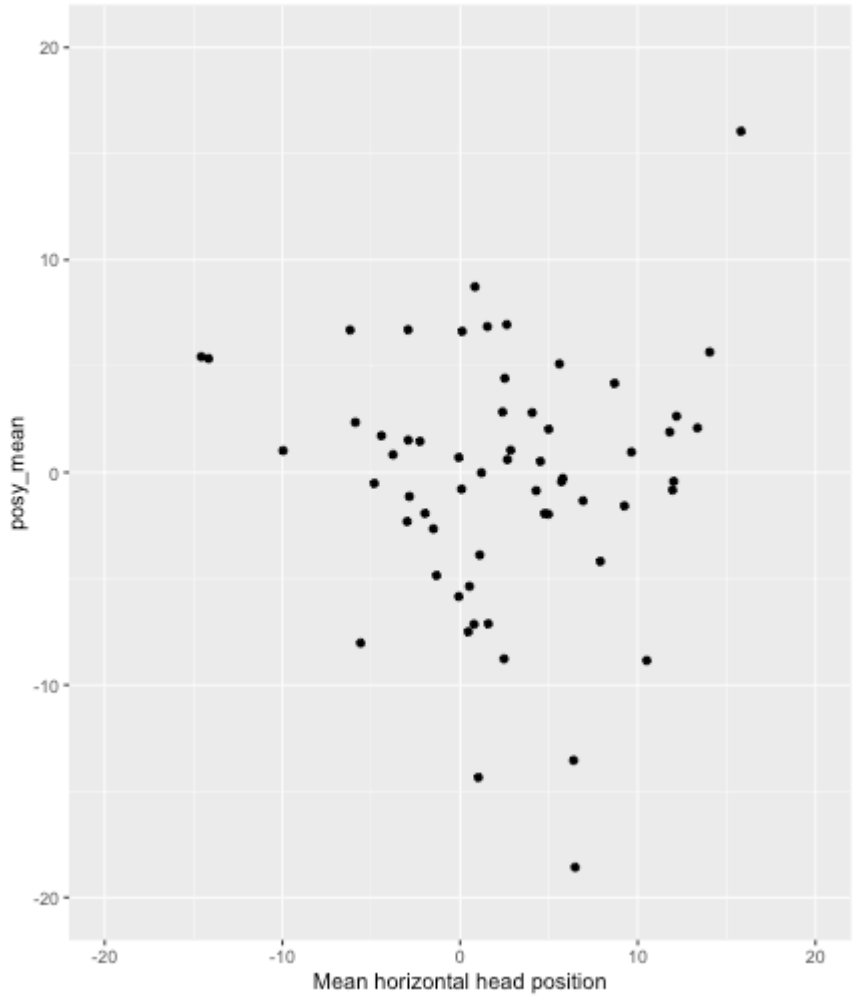
Simple scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) +
  geom_point() +
  xlim(-20, 20) +
  ylim(-20, 20)
```



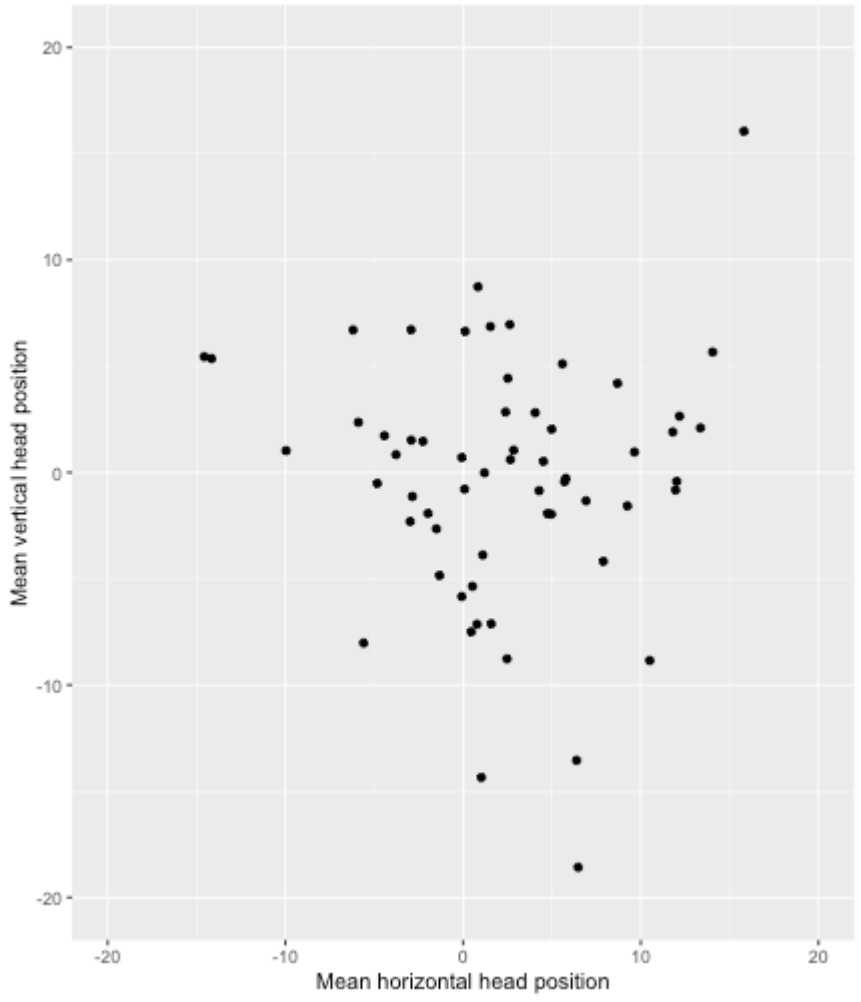
Simple scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) +
  geom_point() +
  xlim(-20, 20) +
  ylim(-20, 20) +
  xlab("Mean horizontal head position")
```



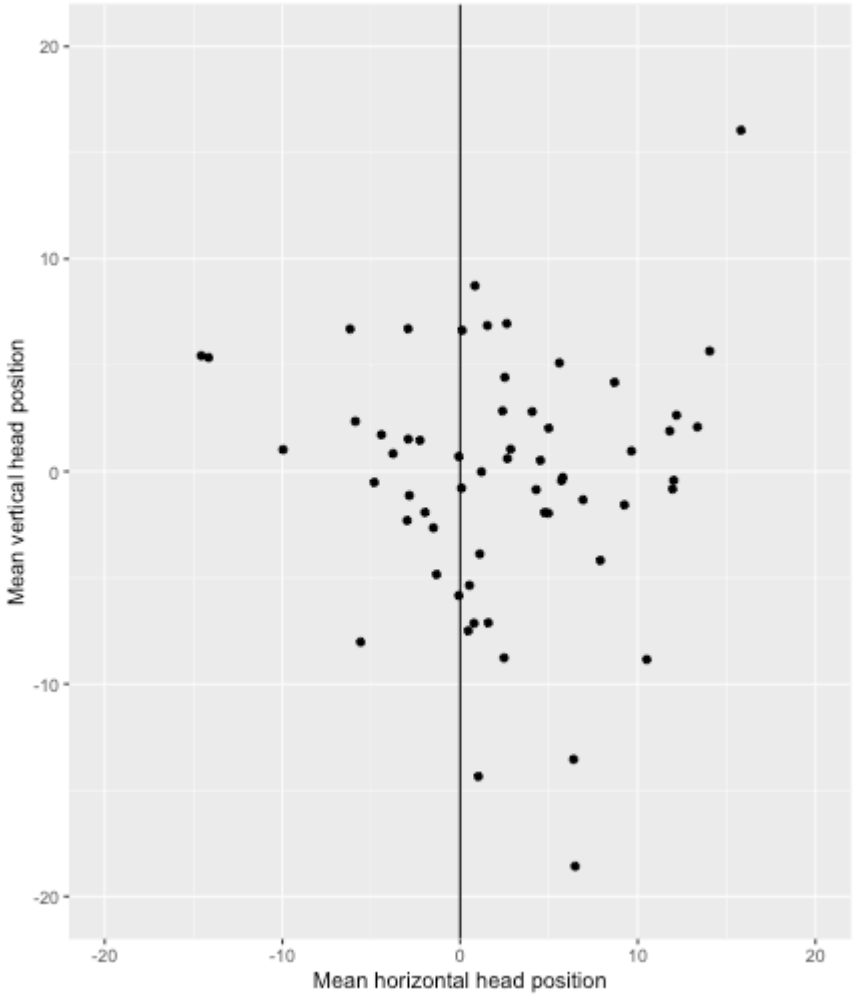
Simple scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) +
  geom_point() +
  xlim(-20, 20) +
  ylim(-20, 20) +
  xlab("Mean horizontal head position") +
  ylab("Mean vertical head position")
```



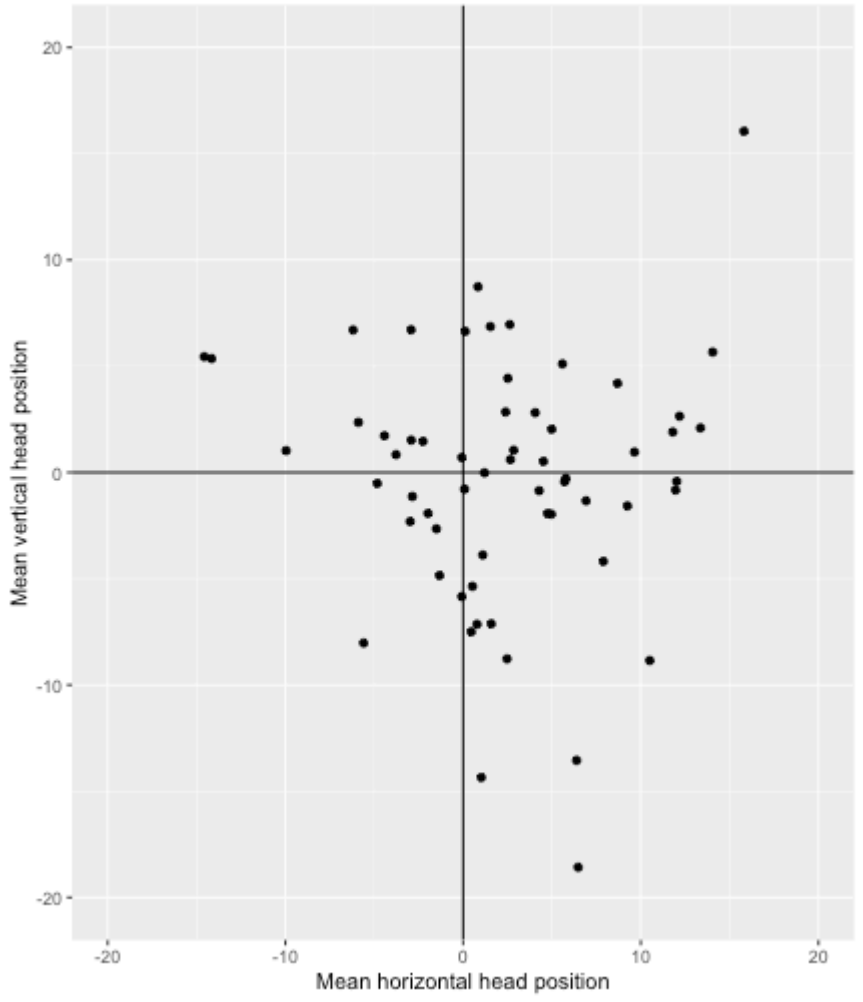
Simple scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) +
  geom_point() +
  xlim(-20, 20) +
  ylim(-20, 20) +
  xlab("Mean horizontal head position") +
  ylab("Mean vertical head position") +
  geom_vline(xintercept = 0)
```



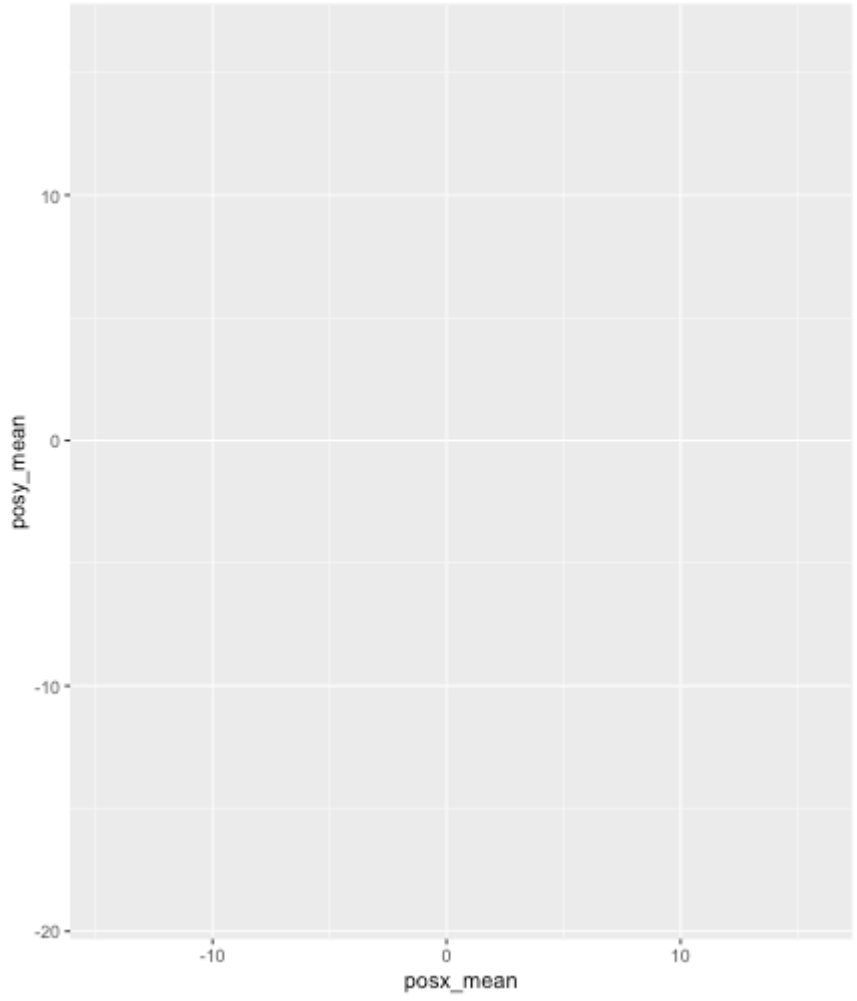
Simple scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) +
  geom_point() +
  xlim(-20, 20) +
  ylim(-20, 20) +
  xlab("Mean horizontal head position") +
  ylab("Mean vertical head position") +
  geom_vline(xintercept = 0) +
  geom_hline(yintercept = 0)
```



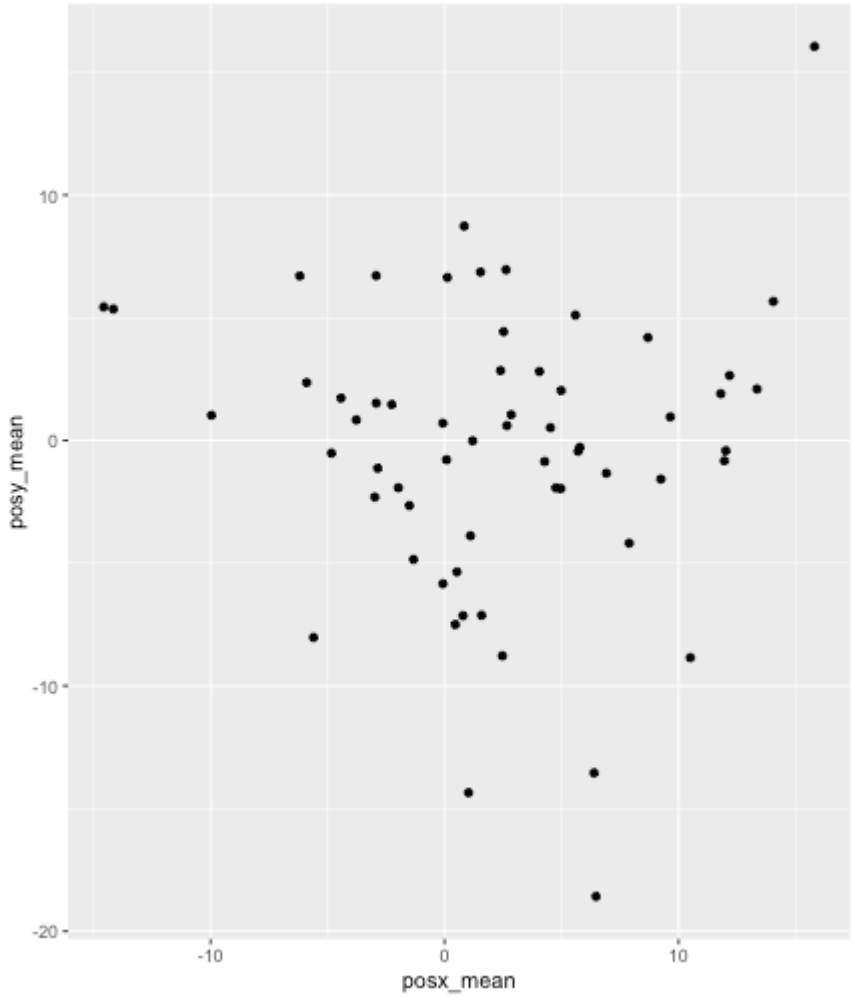
Facetting a scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean))
```



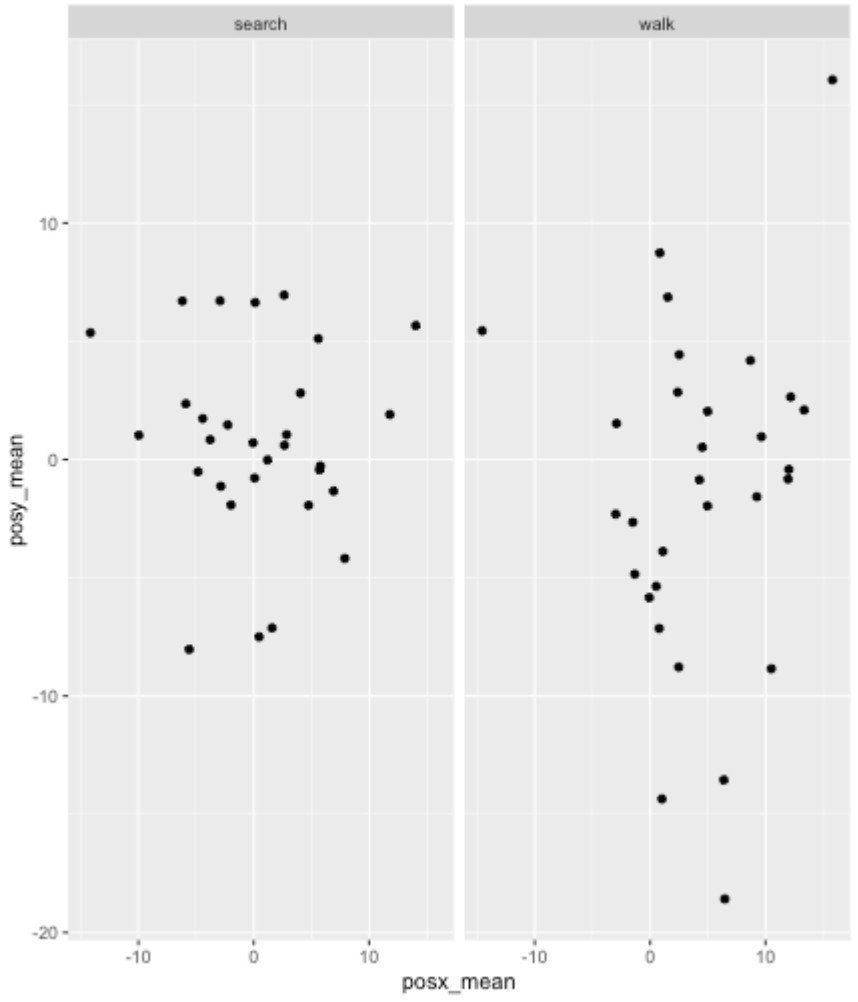
Facetting a scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) +
  geom_point()
```



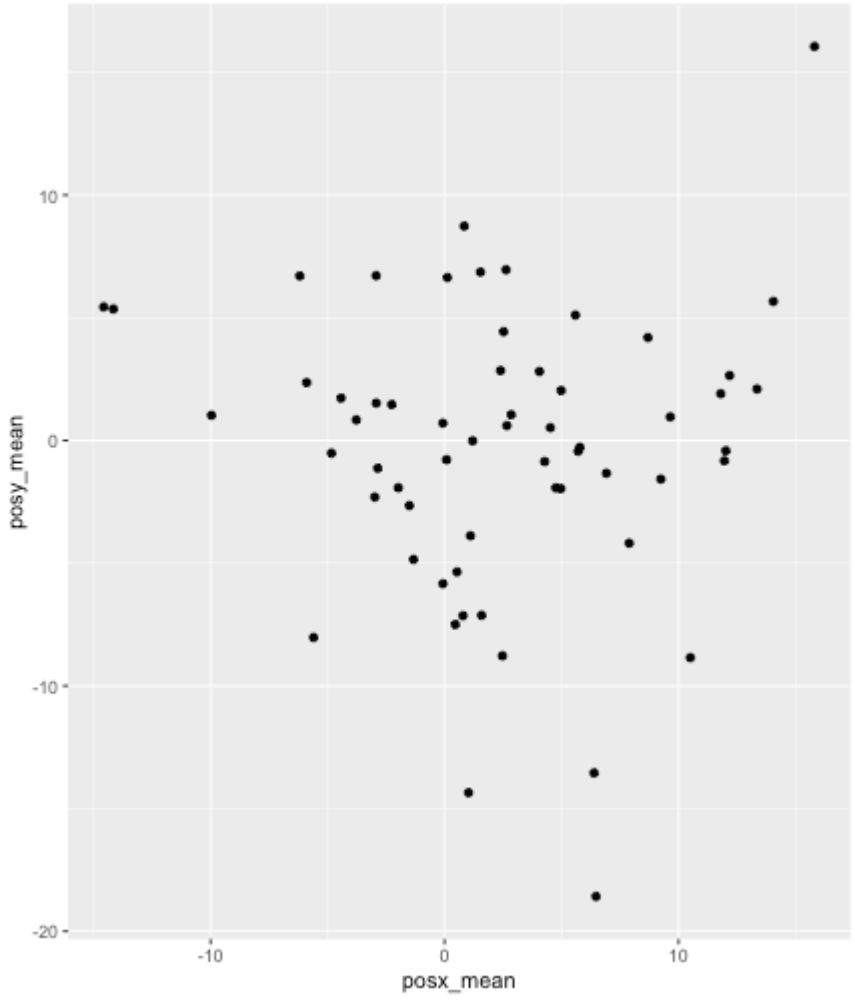
Facetting a scatterplot

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) +
  geom_point() +
  facet_wrap("condition", ncol = 2)
```



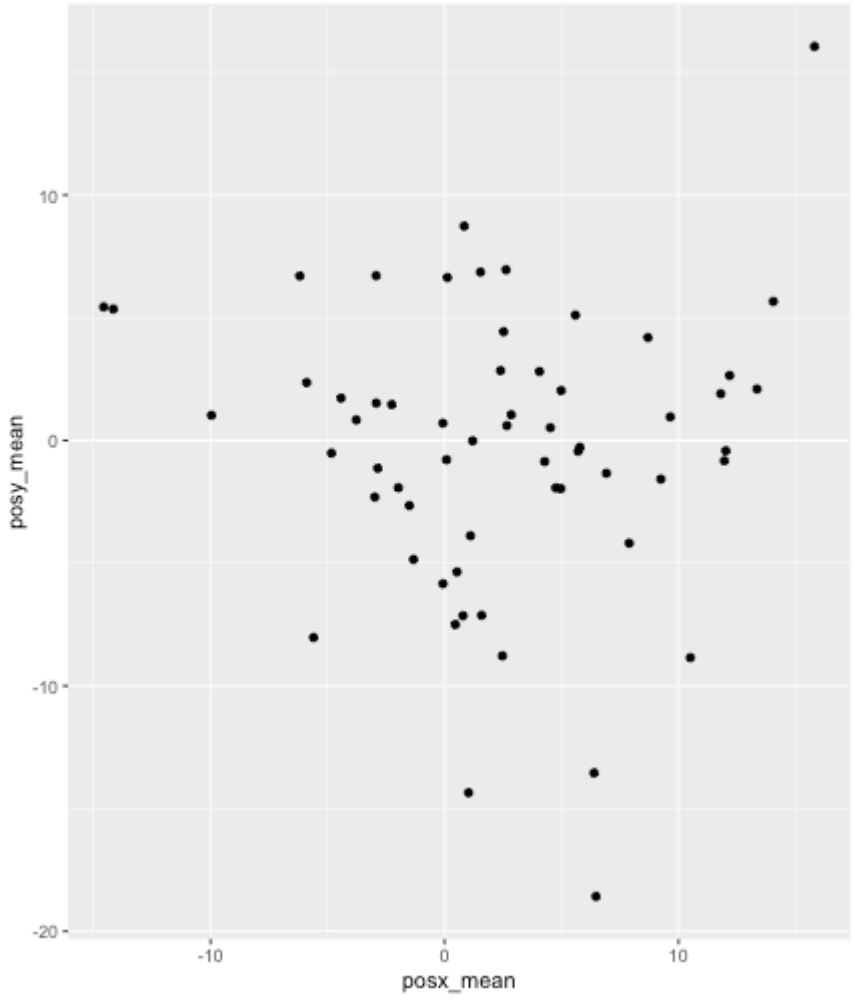
Saving plots

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) + geom_point()
```



Saving plots

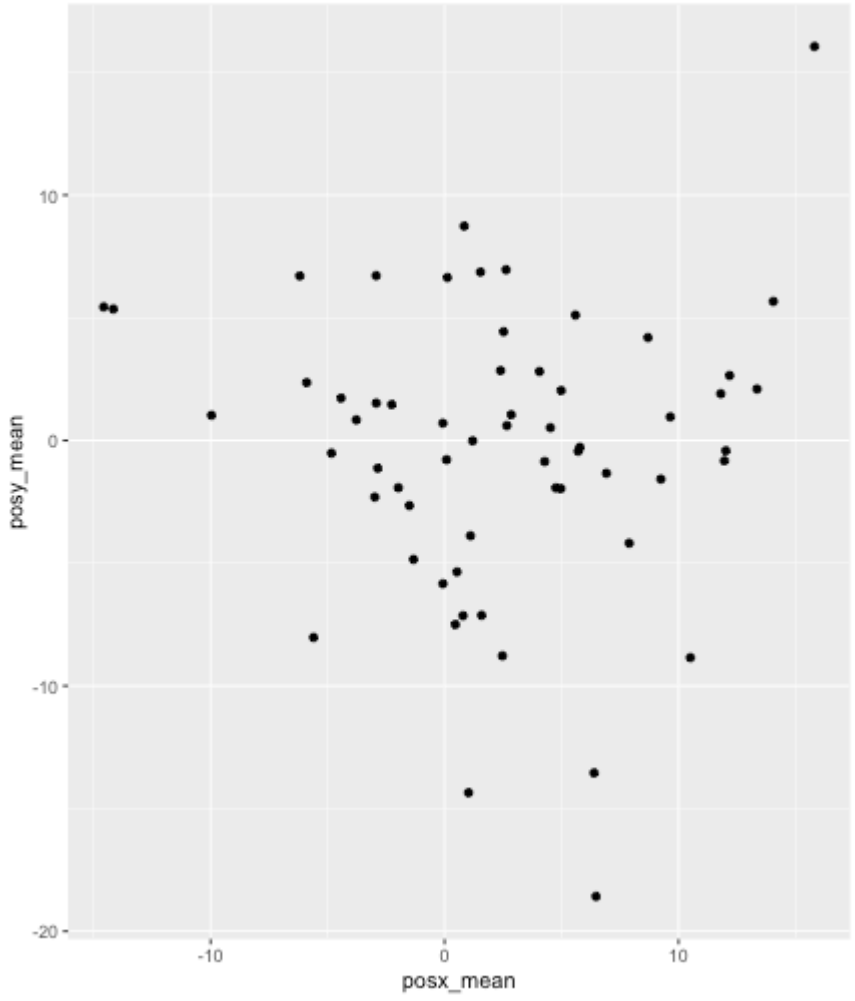
```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) + geom_point()
#ggsave will save the last plot to file
ggsave("eda/head-position-scatter.jpg")
```



Saving plots

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) + geom_point()
#ggsave will save the last plot to file
ggsave("eda/head-position-scatter.jpg")

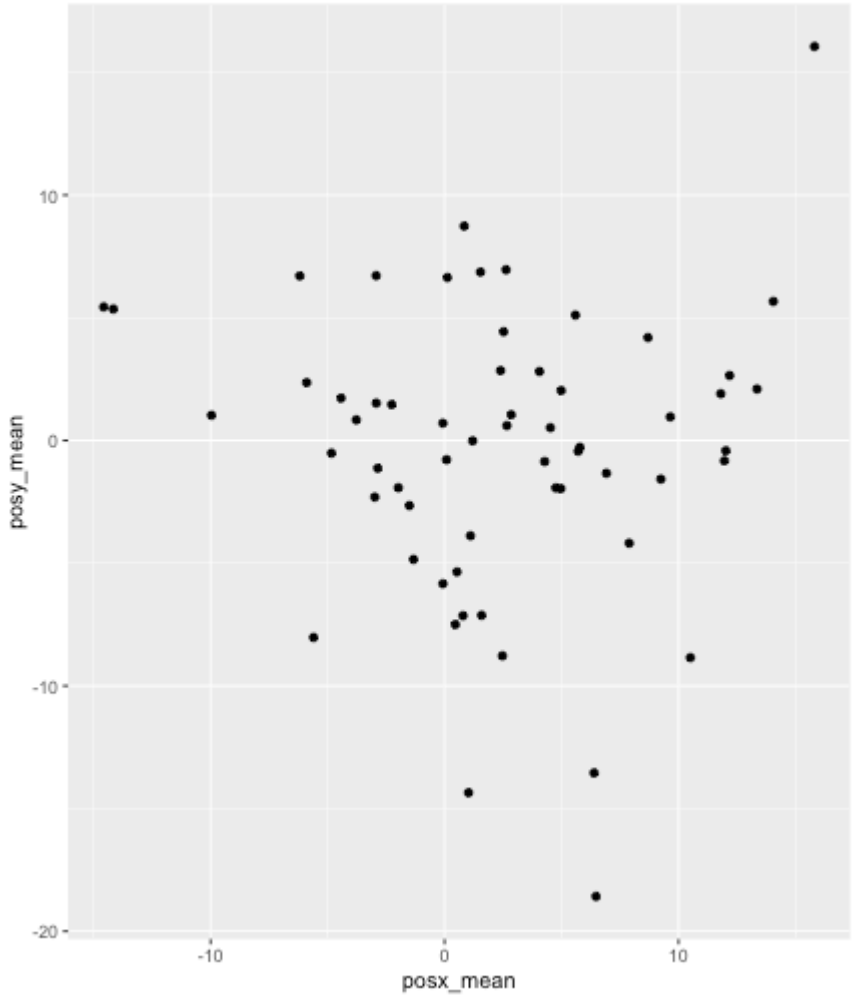
#You can also save plots to objects (they are lists)  ou can also save
```



Saving plots

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) + geom_point()
#ggsave will save the last plot to file
ggsave("eda/head-position-scatter.jpg")

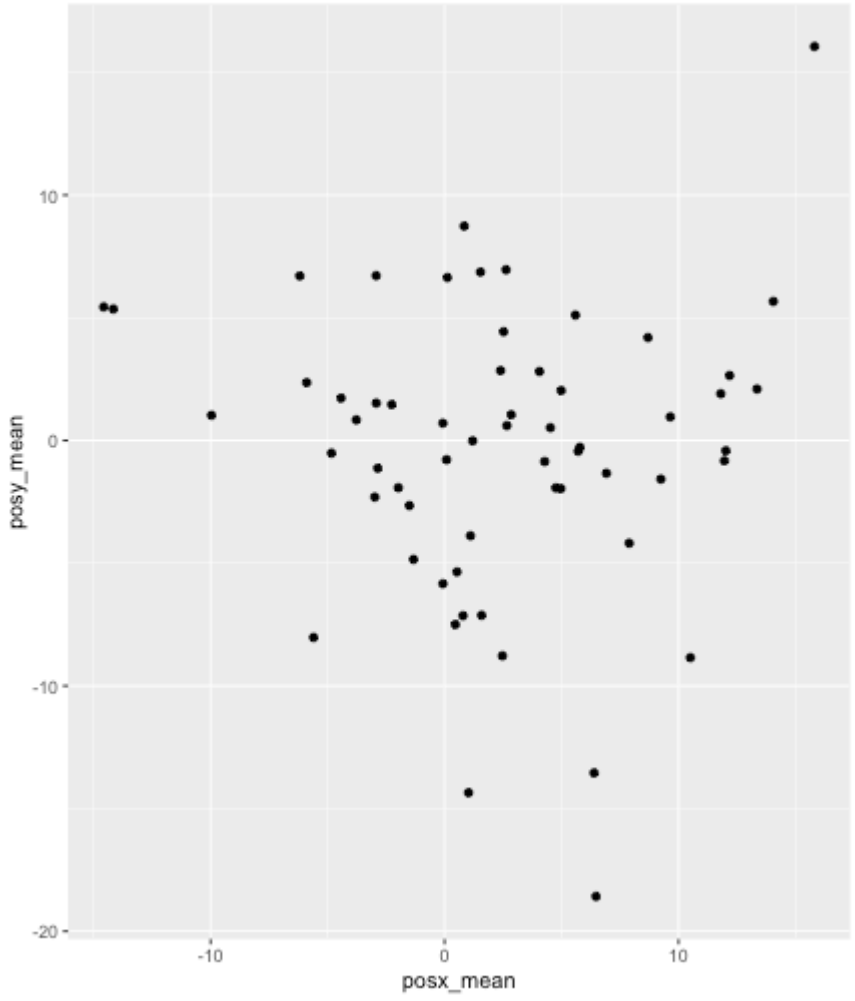
#You can also save plots to objects (they are lists)  ou can also save
p1 <- ggplot(ds, aes(x = posx_mean, y = posy_mean)) + geom_point()
```



Saving plots

```
ggplot(ds, aes(x = posx_mean, y = posy_mean)) + geom_point()
#ggsave will save the last plot to file
ggsave("eda/head-position-scatter.jpg")

#You can also save plots to objects (they are lists)  ou can also save
p1 <- ggplot(ds, aes(x = posx_mean, y = posy_mean)) + geom_point()
ggsave("eda/head-position-scatter.jpg", plot = p1)
```



Useful ggsave options

- `width`, `height`, and `units` = specify the size of the saved plot
- `device` = "png", "eps", "pdf" (but will choose based on filename)
- `scale` = bump up the scale (1.5) if the text/lines look too big or down (.5) if text/lines look too small

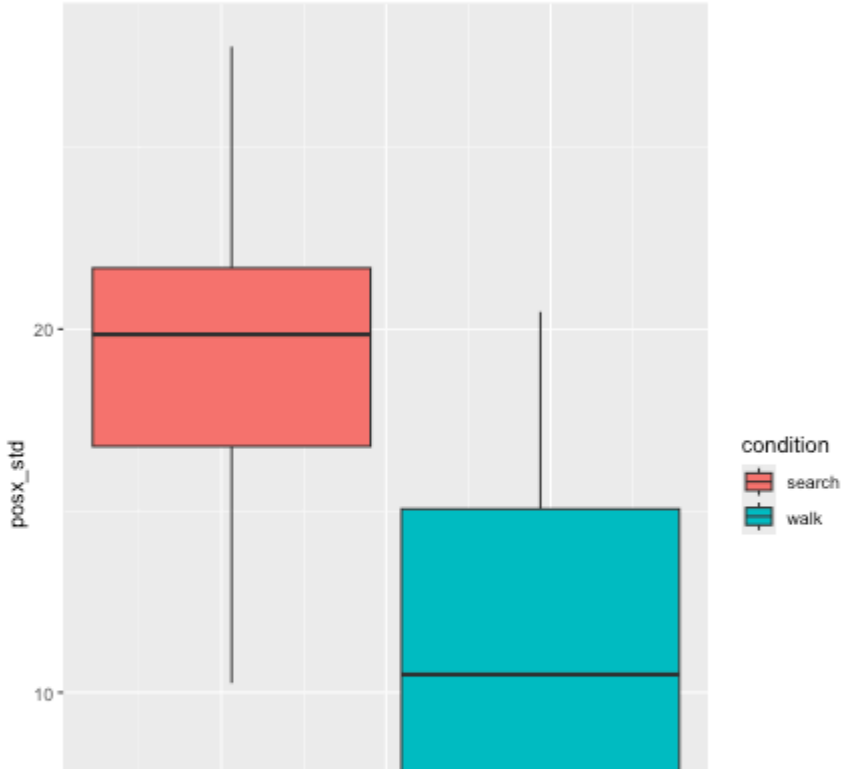
```
ggsave("eda/output.pdf", units = "in", height = 4, width = 4,
scale = .75)
```

Automation + plotting

- Each plot you create appears in the plots tab in RStudio
- Flipping through a few plots is OK, but not hundreds
- Write plotting functions that can save plots to file to better organize them

Automation + plotting

```
plot_dv_by_condition <- function(dv) {
  ggplot(ds, aes(fill = condition, y = .data[[dv]])) +
    geom_boxplot()
}
plot_dv_by_condition("posx_std")
```



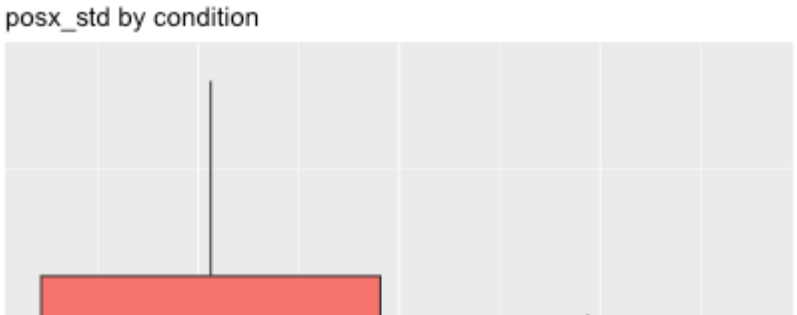
Why does that work?

- `ds %>% .[["posx_std"]]` is the same as `ds$posx_std`
- Double brackets let you access the column by character name
- `.data` is a ggplot shortcut for 'the current data'
- `y = .data[[dv]]` says 'use the name in dv to choose the y column'

Make the function more useful with labels/saving

```
plot_dv_by_condition <- function(dv) {
  p <- ggplot(ds, aes(fill = condition, y = .data[[dv]])) +
    geom_boxplot() +
    ylab(dv) + #we can also use dv to label the axis
    ggtitle(str_glue("{dv} by condition")) + #and title the plot
    theme(legend.position = "bottom")

  # and we can use it to write a sensible output filename
  ggsave(str_glue("eda/by_condition/{dv}_by_condition_boxplot.jpg"), plot = p,
         height = 6, width = 4, units = "cm", scale = 2)
  return(p)
}
plot_dv_by_condition("posx_std")
```



Use colnames() to get each of the dv characters

```
dvs_to_plot <- ds %>% select(-(id:humidity)) %>% colnames
dvs_to_plot
```

```
 [1] "len"             "gazex_std"       "gazey_std"       "posx_mean"
 [5] "posx_std"        "posx_speed"      "posy_mean"       "posy_std"
 [9] "posy_speed"      "head_speed"      "eyex_mean"       "eyex_std"
[13] "eyex_speed"      "eyey_mean"       "eyey_std"        "eyey_speed"
[17] "eye_speed"       "eyexpos_corr"    "eyeypos_corr"    "bothcent"
[21] "eyecent"         "headcent"        "nocent_samedir"  "nocent_oppdir"
[25] "path_total"      "path_speed"      "path_speed_sd"   "straightness"
[29] "eyexspeed_corr"  "headxspeed_corr" "gazexspeed_corr"
```

```
walk(dvs_to_plot, plot_dv_by_condition)
```

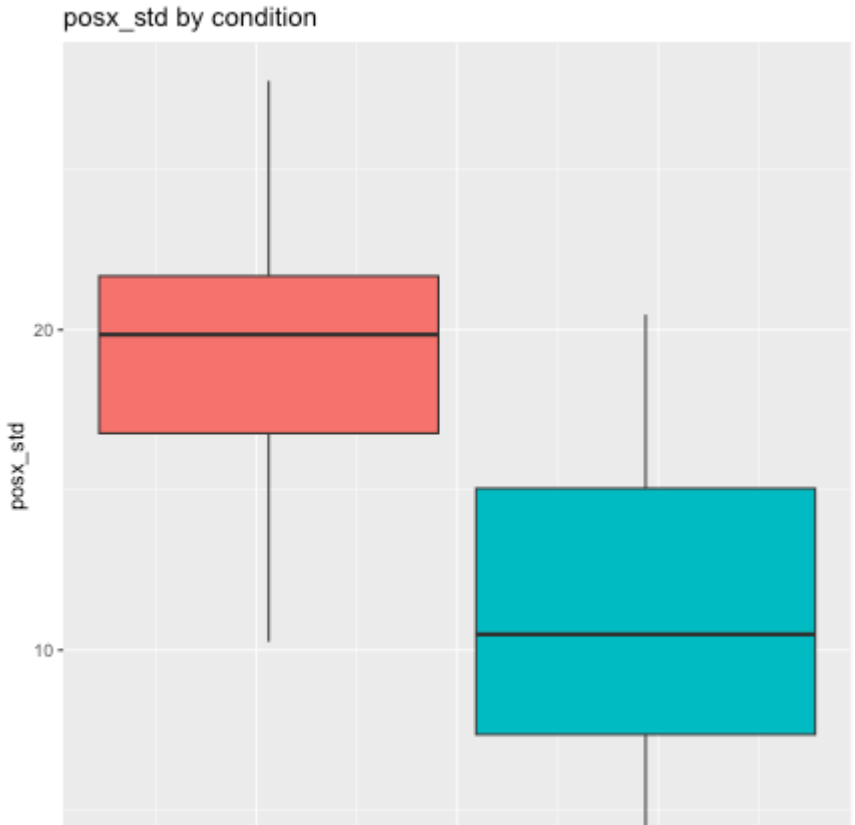
What if we want to specify the type of plot?

```
plot_dv_by_condition <- function(dv, geom = "boxplot") {
  # Start of the ggplot
  p <- ggplot(ds) +
    ylab(dv) +
    ggtitle(str_glue("{dv} by condition")) +
    theme(legend.position = "bottom")

  # Add the geom based on the text argument
  if (geom == "boxplot") p <- p + geom_boxplot(aes(fill = condition, y = .data[[dv]]))
  if (geom == "density") p <- p + geom_density(aes(color = condition, x = .data[[dv]]))
  ggsave(str_glue("eda/by_condition/{dv}_by_condition_{geom}.jpg"), plot = p,
         height = 6, width = 4, units = "cm", scale = 2)
  return(p)
}
```

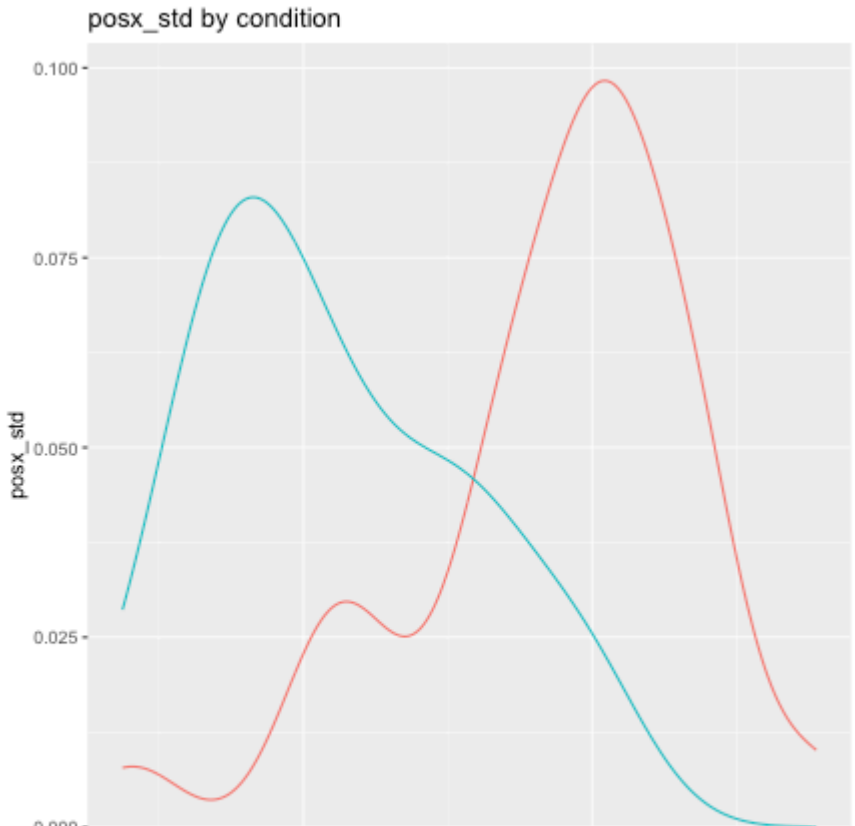
What if we want to specify the type of plot?

```
plot_dv_by_condition(dv = "posx_std", geom = "boxplot")
```



What if we want to specify the type of plot?

```
plot_dv_by_condition(dv = "posx_std", geom = "density")
```



What if we want to plot subsets of data?

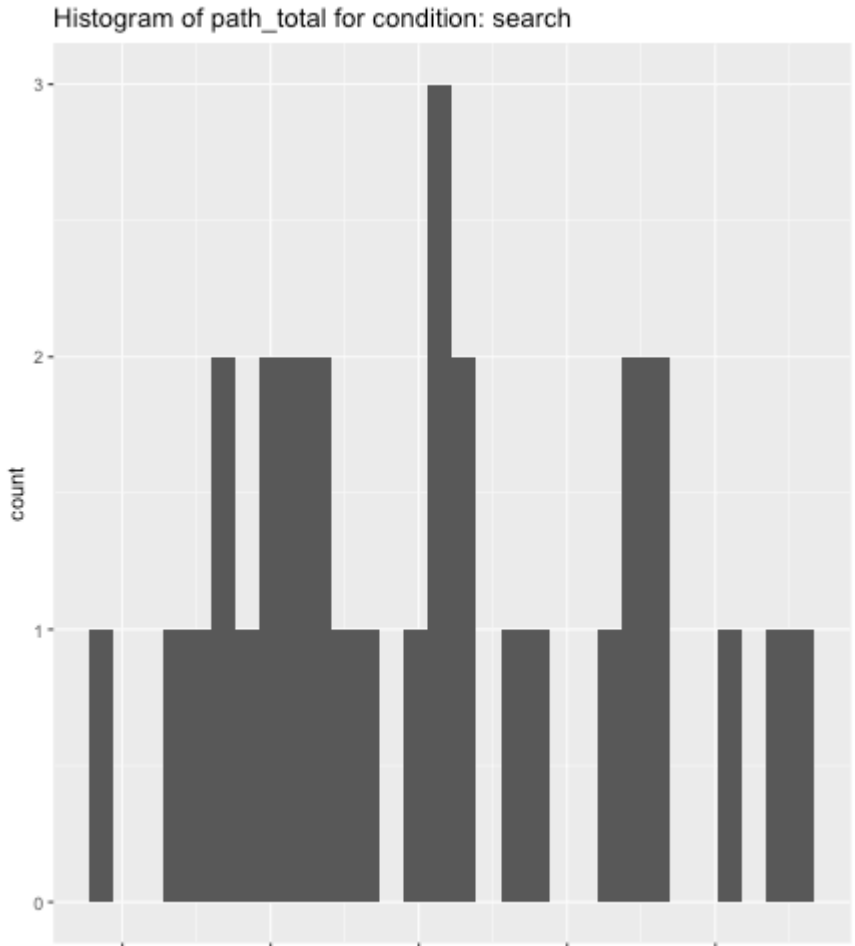
```
plot_filtered_histogram <- function(dv, cond = "All") {
  # Use an if/else to create a subset of data if condition is provided
  if (cond == "All") {
    temp_data <- ds
  } else {
    temp_data <- ds %>% filter(condition == cond)
  }

  # Plot based on the subset
  p <- ggplot(temp_data, aes(x = .data[[dv]])) +
    geom_histogram() +
    ggtitle(str_glue("Histogram of {dv} for condition: {cond}"))

  return(p)
}
```

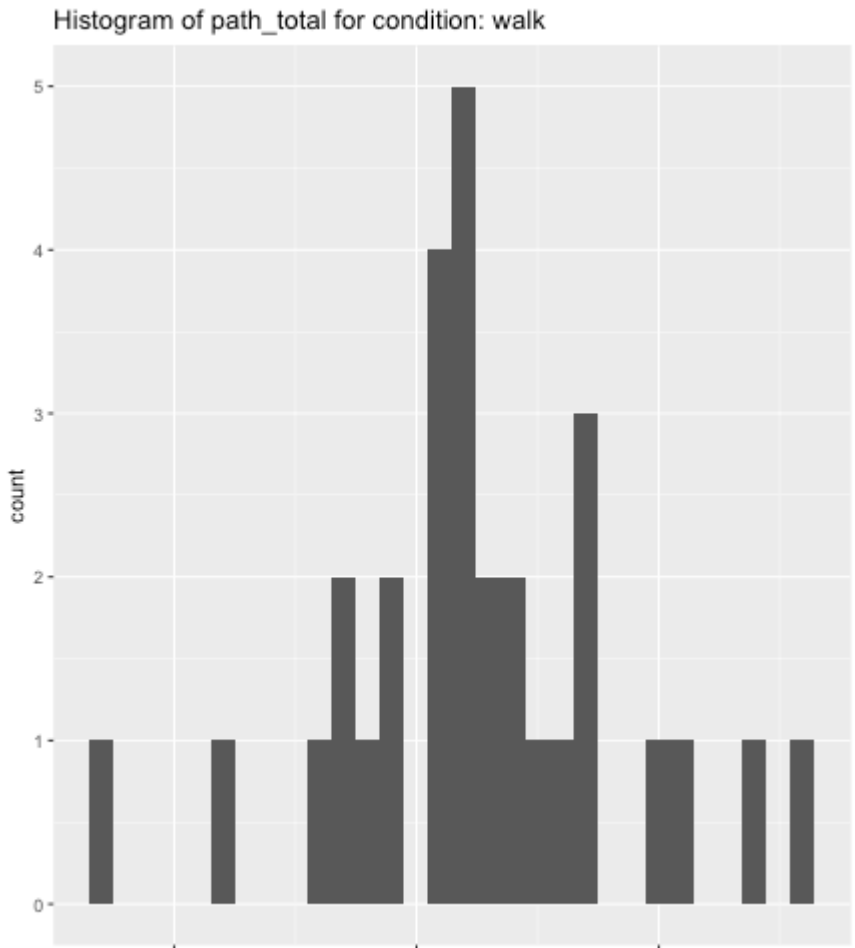
What if we want to plot subsets of data?

```
plot_filtered_histogram(dv = "path_total", cond = "search")
```



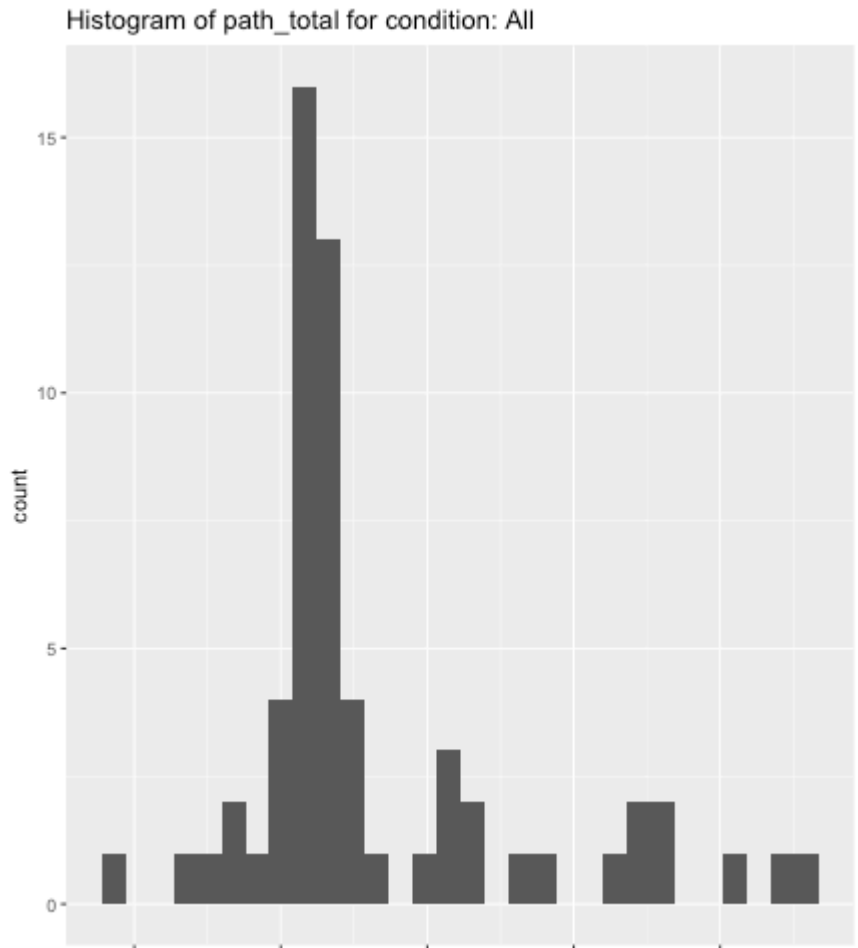
What if we want to plot subsets of data?

```
plot_filtered_histogram(dv = "path_total", cond = "walk")
```



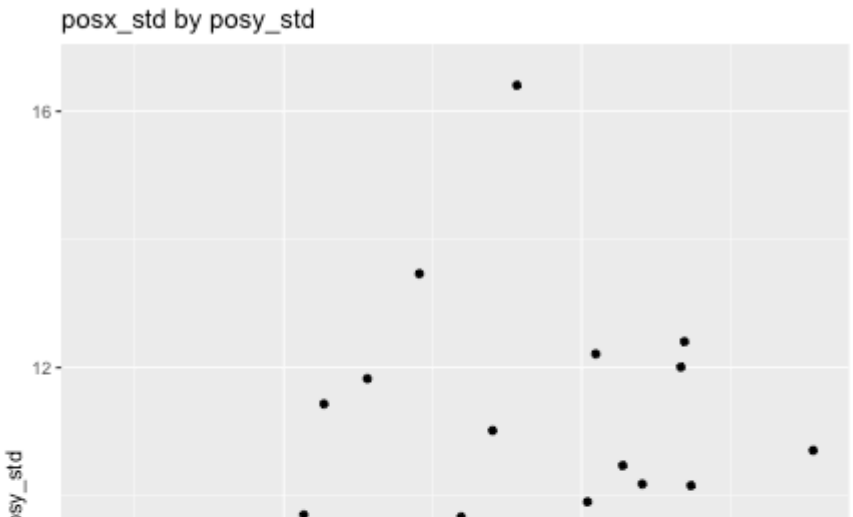
What if we want to plot subsets of data?

```
plot_filtered_histogram(dv = "path_total")
```



Two variable inputs

```
two_var_scatter <- function(df, var1, var2) {
  p <- ggplot(df, aes(x = .data[[var1]], y = .data[[var2]])) +
    geom_point() +
    xlab(var1) +
    ylab(var2) +
    ggtitle(str_glue("{var1} by {var2}"))
  ggsave(str_glue("eda/scatter/{var1}_{var2}.jpg"), plot = p,
         height = 7, width = 7, units = "cm", scale = 1.5)
  return(p)
}
two_var_scatter(ds, "posx_std", "posy_std")
```



Get x and y pairs from colnames

```
pred_x <- ds %>% select(matches("x_")) %>% colnames
pred_x
```

```
[1] "gazex_std"  "posx_mean"  "posx_std"   "posx_speed" "eyex_mean"
[6] "eyex_std"   "eyex_speed"
```

```
pred_y <- ds %>% select(matches("y_") & where(is.numeric)) %>% colnames
pred_y
```

```
[1] "gazey_std"  "posy_mean"  "posy_std"   "posy_speed" "eyey_mean"
[6] "eyey_std"   "eyey_speed"
```

```
pred_x_by_y <- map2(pred_x, pred_y, ~ two_var_scatter(ds, .x, .y))
pred_x_by_y
```

```
[[1]]
```



Pivoting and faceting to automate plots

```
ds_long <- ds %>% select(matches("x_")) %>%
  pivot_longer(cols = everything())

slice_head(ds_long, n = 7)
```

```
# A tibble: 7 × 2
  name        value
  <chr>       <dbl>
1 gazex_std  15.6
2 posx_mean   8.69
3 posx_std    6.13
4 posx_speed  0.470
5 eyex_mean  -0.798
6 eyex_std   11.5
7 eyex_speed  1.79
```

Pivoting and faceting to automate plots

```
ds_long %>% ggplot(aes(x = value)) +
  geom_histogram() +
  facet_wrap(~ name, scales = "free", nrow = 2)
```

